

Unit 3

MEMORY MANAGEMENT

Lecture #15

Inter-process Communication (IPC)

- Mechanism for processes to communicate and to synchronize their actions.
- Message system – processes communicate with each other without resorting to shared variables.
- IPC facility provides two operations:
 1. **send**(*message*) – message size fixed or variable
 2. **receive**(*message*)
- If *P* and *Q* wish to communicate, they need to:
 1. establish a *communication link* between them
 2. exchange messages via send/receive
- Implementation of communication link
 1. physical (e.g., shared memory, hardware bus)
 2. logical (e.g., logical properties)

Direct Communication

Processes must name each other explicitly:

- ♦ **send** (*P*, *message*) – send a message to process *P*
- ♦ **receive**(*Q*, *message*) – receive a message from process *Q* Properties of communication link
- ♦ Links are established automatically.
- ♦ A link is associated with exactly one pair of communicating processes.
- ♦ Between each pair there exists exactly one link.
- ♦ The link may be unidirectional, but is usually bi-directional.

Indirect Communication

Messages are directed and received from mailboxes (also referred to as ports). ♦

Each mailbox has a unique id.

- ♦ Processes can communicate only if they share a mailbox.

Properties of communication link

- ♦ Link established only if processes share a common mailbox
- ♦ A link may be associated with many processes.
- ♦ Each pair of processes may share several communication links.
- ♦ Link may be unidirectional or bi-directional.

Operations

- ♦ create a new mailbox
- ♦ send and receive messages through mailbox
- ♦ destroy a mailbox

Primitives are defined as:

send(*A*, *message*) – send a message to mailbox *A* **receive**(*A*,
message) – receive a message from mailbox *A*

Mailbox sharing

- ♦ *P1*, *P2*, and *P3* share mailbox *A*.
- ♦ *P1*, sends; *P2* and *P3* receive. ♦ Who gets the message?

Solutions

- ♦ Allow a link to be associated with at most two processes.

- ◆ Allow only one process at a time to execute a receive operation. ◆ Allow the system to select arbitrarily the receiver. Sender is notified who the receiver was.

Concurrent Processes

The concurrent processes executing in the operating system may be either independent processes or cooperating processes. A process is independent if it cannot affect or be affected by the other processes executing in the system. Clearly, any process that does not share any data (temporary or persistent) with any other process is independent. On the other hand, a process is cooperating if it can affect or be affected by the other processes executing in the system. Clearly, any process that shares data with other processes is a cooperating process.

We may want to provide an environment that allows process cooperation for several reasons:

Information sharing: Since several users may be interested in the same piece of information (for instance, a shared file), we must provide an environment to allow concurrent access to these types of resources.

Computation speedup: If we want a particular task to run faster, we must break it into subtasks, each of which will be executing in parallel with the others. Such a speedup can be achieved only if the computer has multiple processing elements (such as CPUs or I/O channels).

Modularity: We may want to construct the system in a modular fashion, dividing the system functions into separate processes or threads.

Convenience: Even an individual user may have many tasks on which to work at one time. For instance, a user may be editing, printing, and compiling in parallel.

Lecture # 16

Message-Passing System

The function of a message system is to allow processes to communicate with one another without the need to resort to shared data. We have already seen message passing used as a method of communication in microkernels. In this scheme, services are provided as ordinary user processes. That is, the services operate outside of the kernel. Communication among the user processes is accomplished through the passing of messages. An IPC facility provides at least the two operations: send (message) and receive (message).

Messages sent by a process can be of either fixed or variable size. If only fixed-sized messages can be sent, the system-level implementation is straightforward. This restriction, however, makes the task of programming more difficult. On the other hand, variable-sized messages require a more complex system-level implementation, but the programming task becomes simpler.

If processes P and Q want to communicate, they must send messages to and receive messages from each other; a communication link must exist between them. This link can be implemented in a variety of ways. We are concerned here not with the link's physical implementation, but rather with its logical implementation. Here are several methods for logically implementing a link and the send/receive operations:

- Direct or indirect communication
- Symmetric or asymmetric communication
- Automatic or explicit buffering
- Send by copy or send by reference
- Fixed-sized or variable-sized messages

We look at each of these types of message systems next.

Synchronization

Communication between processes takes place by calls to send and receive primitives. There are different design options for implementing each primitive. Message passing may be either blocking or nonblocking-also known as synchronous and asynchronous.

Blocking send: The sending process is blocked until the message is received by the receiving process or by the mailbox.

Nonblocking send: The sending process sends the message and resumes operation.

Blocking receive: The receiver blocks until a message is available.

Nonblocking receive: The receiver retrieves either a valid message or a null.

Different combinations of send and receive are possible. When both the send and receive are blocking, we have a rendezvous between the sender and the receiver.

Buffering

Whether the communication is direct or indirect, messages exchanged by communicating processes reside in a temporary queue. Basically, such a queue can be implemented in three ways:

Zero capacity: The queue has maximum length 0; thus, the link cannot have any messages waiting in it. In this case, the sender must block until the recipient receives the message.

Bounded capacity: The queue has finite length n ; thus, at most n messages can reside in it. If the queue is not full when a new message is sent, the latter is placed in the queue (either the message is copied or a pointer to the message is kept), and the sender can continue execution without waiting. The link has a finite capacity, however. If the link is full, the sender must block until space is available in the queue.

Unbounded capacity: The queue has potentially infinite length; thus, any number of messages can wait in it. The sender never blocks. The zero-capacity case is sometimes referred to as a message system with no buffering; the other cases are referred to as automatic buffering.

Client-Server Communication

- Sockets
- Remote Procedure Calls
- Remote Method Invocation (Java)

Lecture # 17

Process Synchronization

A situation where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**.

Producer-Consumer Problem

Paradigm for cooperating processes, *producer* process produces information that is consumed by a *consumer* process.

To allow producer and consumer processes to run concurrently, we must have available a buffer of items that can be filled by the producer and emptied by the consumer. A producer can produce one item while the consumer is consuming another item. The producer and consumer must be synchronized, so that the consumer does not try to consume an item that has not yet been produced. In this situation, the consumer must wait until an item is produced.

♦ *unbounded-buffer* places no practical limit on the size of the buffer. ♦
bounded-buffer assumes that there is a fixed buffer size.

Bounded-Buffer – Shared-Memory Solution

The consumer and producer processes share the following variables.

Shared data

```
#define BUFFER_SIZE 10
typedef struct
{
    ...
} item;
item buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
```

Solution is correct, but can only use BUFFER_SIZE-1 elements.

The shared buffer is implemented as a circular array with two logical pointers: in and out. The variable in points to the next free position in the buffer; out points to the first full position in the buffer. The buffer is empty when $in == out$; the buffer is full when $((in + 1) \% BUFFERSIZE) == out$.

The code for the producer and consumer processes follows. The producer process has a local variable **nextproduced** in which the new item to be produced is stored:

Bounded-Buffer – Producer Process

```
item nextProduced;
while (1)
{
    while (((in + 1) \% BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] =
    nextProduced;
    in = (in + 1) \% BUFFER_SIZE;
}
```

Bounded-Buffer – Consumer Process

```
item nextConsumed;
while (1)
{
while (in == out) ; /* do
nothing */ nextConsumed =
buffer[out];
out = (out + 1) % BUFFER_SIZE;
}
```

The critical section problem

Consider a system consisting of n processes $\{P_0, P_1, \dots, P_{n-1}\}$. Each process has a segment of code, called a **critical section**, in which the process may be changing common variables, updating a table, writing a file, and so on. The important feature of the system is that, when one process is executing in its critical section, no other process is to be allowed to execute in its critical section. Thus, the execution of critical sections by the processes is **mutually exclusive** in time. The critical-section problem is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section. The section of code implementing this request is the entry section. The critical section may be followed by an exit section. The remaining code is the remainder section.

```
do{
    Entry section
        Critical section
    Exit section
        Remainder section
}while(1);
```

A solution to the critical-section problem must satisfy the following three requirements:

1. **Mutual Exclusion:** If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
2. **Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder section can participate in the decision on which will enter its critical section next, and this selection cannot be postponed indefinitely.
3. **Bounded Waiting:** There exists a bound on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Lecture # 18

Peterson's solution

Peterson's solution is a software based solution to the critical section problem.

Consider two processes P0 and P1. For convenience, when presenting Pi, we use Pi to denote the other process; that is, $j = 1 - i$.

The processes share two variables:

boolean flag [2] ; int turn;

Initially flag [0] = flag [1] = false, and the value of turn is immaterial (but is either 0 or 1).

The structure of process Pi is shown below.

```
do{
    flag[i]=true
    turn=j
    while(flag[j] && turn==j);
        critical        section
    flag[i]=false
        Remainder section
}while(1);
```

To enter the critical section, process Pi first sets flag [i] to be true and then sets turn to the value j, thereby asserting that if the other process wishes to enter the critical section it can do so. If both processes try to enter at the same time, turn will be set to both i and j at roughly the same time. Only one of these assignments will last; the other will occur, but will be overwritten immediately. The eventual value of turn decides which of the two processes is allowed to enter its critical section first.

We now prove that this solution is correct. We need to show that:

1. Mutual exclusion is preserved,
2. The progress requirement is satisfied,
3. The bounded-waiting requirement is met.

To prove property 1, we note that each Pi enters its critical section only if either flag [j] == false or turn == i. Also note that, if both processes can be executing in their critical sections at the same time, then flag [i] == flag [j] == true. These two observations imply that P0 and P1 could not have successfully executed their while statements at about the same time, since the value of turn can be either 0 or 1, but cannot be both. Hence, one of the processes say Pj-must have successfully executed the while statement, whereas Pi had to execute at least one additional statement ("turn == j"). However, since, at that time, flag [j] == true, and turn == j, and this condition will persist as long as Pi is in its critical section, the result follows:

To prove properties 2 and 3, we note that a process Pi can be prevented from entering the critical section only if it is stuck in the while loop with the condition flag [j] == true and turn == j; this loop is the only one. If Pi is not ready to enter the critical section, then flag [j] == false and Pi can enter its critical section. If Pi has set flag[j] to true and is also executing in its while statement, then either turn == i or turn == j. If turn == i, then Pi will enter the critical section. If turn == j, then Pi will enter the critical section. However, once Pi exits its critical section, it will reset flag [j] to false, allowing Pi to enter its critical section. If Pi resets flag [j] to true, it must also set turn to i.

Thus, since Pi does not change the value of the variable turn while executing the while statement, Pi will enter the critical section (progress) after at most one entry by Pi (bounded waiting).

Lecture # 19

Synchronization Hardware

As with other aspects of software, hardware features can make the programming task easier and improve system efficiency. In this section, we present some simple hardware instructions that are available on many systems, and show how they can be used effectively in solving the critical-section problem.

The definition of the TestAndSet instruction.

```
boolean TestAndSet(boolean &target)
{
    boolean rv = target;
    target = true;
    return rv;
}
```

The critical-section problem could be solved simply in a uniprocessor environment if we could forbid interrupts to occur while a shared variable is being modified. In this manner, we could be sure that the current sequence of instructions would be allowed to execute in order without preemption. No other instructions would be run, so no unexpected modifications could be made to the shared variable.

Unfortunately, this solution is not feasible in a multiprocessor environment. Disabling interrupts on a multiprocessor can be time-consuming, as the message is passed to all the processors. This message passing delays entry into each critical section, and system efficiency decreases. Also, consider the effect on a system's clock, if the clock is kept updated by interrupts.

Many machines therefore provide special hardware instructions that allow us either to test and modify the content of a word, or to swap the contents of two words, atomically—that is, as one uninterruptible unit. We can use these special instructions to solve the critical-section problem in a relatively simple manner. Rather than discussing one specific instruction for one specific machine, let us abstract the main concepts behind these types of instructions.

The TestAndSet instruction can be defined as shown in code. The important characteristic is that this instruction is executed atomically. Thus, if two TestAndSet instructions are executed simultaneously (each on a different CPU), they will be executed sequentially in some arbitrary order.

Mutual-exclusion implementation with **TestAndSet** do{

```
    while(!TestAndSet(lock));
    critical section    lock=false
    Remainder section
}while(1);
```

```
void Swap(boolean &a, boolean &b) {
    boolean temp = a; a = b; b = temp;
}
```

If the machine supports the TestAndSet instruction, then we can implement mutual exclusion by declaring a Boolean variable lock, initialized to false.

If the machine supports the Swap instruction, then mutual exclusion can be provided as follows. A global Boolean variable lock is declared and is initialized to false. In addition, each process also has a local Boolean variable key.

Lecture # 20

Semaphores

The solutions to the critical-section problem presented before are not easy to generalize to more complex problems. To overcome this difficulty, we can use a synchronization tool called a semaphore. A **semaphore** *S* is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: **wait** and **signal**. These operations were originally termed *P* (for wait; from the Dutch *proberen*, to test) and *V* (for signal; from *verhogen*, to increment). The classical definition of wait in pseudocode is

```
wait(S)    {
while (S <=
0)
; // no-op
S --
; }
```

The classical definitions of signal in pseudocode is

```
Signal(S){
S++;
}
```

Modifications to the integer value of the semaphore in the wait and signal operations must be executed indivisibly. That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value. In addition, in the case of the wait (*S*), the testing of the integer value of *S* (*S* ≤ 0), and its possible modification (*S*--), must also be executed without interruption.

Usage

We can use semaphores to deal with the *n*-process critical-section problem. The *n* processes share a semaphore, *mutex* (standing for mutual exclusion), initialized to 1. Each process *P_i* is organized as shown in Figure. We can also use semaphores to solve various synchronization problems.

For example, consider two concurrently running processes: *P₁* with a statement *S₁* and *P₂* with a statement *S₂*. Suppose that we require that *S₂* be executed only after *S₁* has completed. We can implement this scheme readily by letting *P₁* and *P₂* share a common semaphore *synch*, initialized to 0, and by inserting the statements in process *P₁*, and the statements

```
wait (synch) ;
s2;
```

in process *P₂*. Because *synch* is initialized to 0, *P₂* will execute *S₂* only after *P₁* has invoked *signal (synch)*, which is after *S₁*.

```
s1;
signal (synch) ; do { wait
(mutex) ; critical section
signal (mutex) ;
remainder section
} while (1);
```

Mutual-exclusion implementation with semaphores.

Implementation

The main disadvantage of the mutual-exclusion solutions and of the semaphore definition given here, is that they all require busy waiting. While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code. This continual looping is clearly a problem in a real multiprogramming system, where a single CPU is

shared among many processes. Busy waiting wastes CPU cycles that some other process might be able to use productively. This type of semaphore is also called a spinlock (because the process "spins" while waiting for the lock). Spinlocks are useful in multiprocessor systems. The advantage of a spinlock is that no context switch is required when a process must wait on a lock, and a context switch may take considerable time. Thus, when locks are expected to be held for short times, spinlocks are useful.

To overcome the need for busy waiting, we can modify the definition of the wait and signal semaphore operations. When a process executes the wait operation and finds that the semaphore value is not positive, it must wait. However, rather than busy waiting, the process can block itself. The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state. Then, control is transferred to the CPU scheduler, which selects another process to execute.

A process that is blocked, waiting on a semaphore S, should be restarted when some other process executes a signal operation. The process is restarted by a wakeup operation, which changes the process from the waiting state to the ready state. The process is then placed in the ready queue. (The CPU may or may not be switched from the running process to the newly ready process, depending on the CPU-scheduling algorithm.)

To implement semaphores under this definition, we define a semaphore as a "C" struct:

```
typedef struct {  
    int value ;  
    struct process  
    *L;  
} semaphore;
```

Each semaphore has an integer value and a list of processes. When a process must wait on a semaphore, it is added to the list of processes. A signal operation removes one process from the list of waiting processes and awakens that process. The wait semaphore operation can now be defined as

```
void wait(semaphore S)  
{ S.value--; if (S.value <  
0) { add this process to  
S . L; block() ;  
}
```

The signal semaphore operation can now be defined as

```
void signal(semaphore S) {  
    S.value++; if (S.value <= 0)  
    {  
        remove a process P from S . L ; wakeup  
(P) ;  
    }
```

The block operation suspends the process that invokes it. The wakeup(P1) operation resumes the execution of a blocked process P. These two operations are provided by the operating system as basic system calls.

Binary Semaphores

The semaphore construct described in the previous sections is commonly known as a counting semaphore, since its integer value can range over an unrestricted domain. A binary semaphore is a semaphore with an integer value that can range only between 0 and 1. A binary semaphore can be simpler to implement than a counting semaphore, depending on the underlying hardware architecture. We will now show how a counting semaphore can be implemented using binary semaphores. Let S be a counting semaphore. To implement it in terms of binary semaphores we need the following data structures:

```
binary-semaphore S1, S2;  
int C;
```

Initially $S1 = 1$, $S2 = 0$, and the value of integer C is set to the initial value of the counting semaphore S .

The wait operation on the counting semaphore S can be implemented as follows:

```
wait (S1)  
; C--; i f  
(C < 0) {  
  signal(S1  
);  
  wait (S2) ;  
}  
signal(S1);
```

The signal operation on the counting semaphore S can be implemented as follows:

```
w a i t (S1) ; C++ ; i f (C <= 0) signal (S2) ; e l s e signal (S1) ;
```

Classic Problems of Synchronization

We present a number of different synchronization problems as examples for a large class of concurrency-control problems. These problems are used for testing nearly every newly proposed synchronization scheme. Semaphores are used for synchronization in our solutions.

The Bounded-Buffer Problem

The bounded-buffer problem is commonly used to illustrate the power of synchronization primitives. We present here a general structure of this scheme, without committing ourselves to any particular implementation. We assume that the pool consists of n buffers, each capable of holding one item. The mutex semaphore provides mutual exclusion for accesses to the buffer pool and is initialized to the value 1. The empty and full semaphores count the number of empty and full buffers, respectively. The semaphore empty is initialized to the value n ; the semaphore full is initialized to the value 0.

The code for the producer process is

```
do{  
  produce an item in nextp  
  ...  
  wait (empty) ;  
  wait (mutex) ;  
  ...  
  add nextp to buffer  
  ...
```

```
signal(mutex); signal (full) ;  
) while (1);
```

The code for the consumer process is

```
do{  
wait (full) ; wait (mutex) ;  
...  
remove an item from buffer to nextc  
...  
signal (mutex) ; signal (empty) ;  
...  
consume the item in nextc  
...  
} while (1);
```

Note the symmetry between the producer and the consumer. We can interpret this code as the producer producing full buffers for the consumer, or as the consumer producing empty buffers for the producer.

Lecture # 22

The Readers- Writers Problem

A data object (such as a file or record) is to be shared among several concurrent processes. Some of these processes may want only to read the content of the shared object, whereas others may want to update (that is, to read and write) the shared object. We distinguish between these two types of processes by referring to those processes that are interested in only reading as readers, and to the rest as writers. Obviously, if two readers access the shared data object simultaneously, no adverse effects will result. However, if a writer and some other process (either a reader or a writer) access the shared object simultaneously, chaos may ensue.

To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared object. This synchronization problem is referred to as the readers-writers problem. Since it was originally stated, it has been used to test nearly every new synchronization primitive. The readers-writers problem has several variations, all involving priorities. The simplest one, referred to as the first readers-writers problem, requires that no reader will be kept waiting unless a writer has already obtained permission to use the shared object. In other words, no reader should wait for other readers to finish simply because a writer is waiting. The second readers-writers problem requires that, once a writer is ready, that writer performs its write as soon as possible. In other words, if a writer is waiting to access the object, no new readers may start reading.

A solution to either problem may result in starvation. In the first case, writers may starve; in the second case, readers may starve. For this reason, other variants of the problem have been proposed. In this section, we present a solution to the first readers-writers problem.

In the solution to the first readers-writers problem, the reader processes share the following data structures:

```
semaphore mutex, wrt;  
int readcount;
```

The semaphores mutex and wrt are initialized to 1; readcount is initialized to 0. The semaphore wrt is common to both the reader and writer processes. The mutex semaphore is used to ensure mutual exclusion when the variable readcount is updated. The readcount variable keeps track of how many processes are currently reading the object. The semaphore wrt functions as a mutual-exclusion semaphore for the writers. It is also used by the first or last reader that enters or exits the critical section. It is not used by readers who enter or exit while other readers are in their critical sections.

The code for a writer process is `do{ wait (wrt) ;`

```
...  
writing is performed  
...  
signal(wrt);  
}while(1);
```

The code for a reader process is

```
do{ wait (mutex) ; readcount++; if (readcount == 1) wait (wrt) ; signal (mutex) ;  
...  
reading is performed  
...  
wait (mutex) ; readcount--; if (readcount == 0) signal(wrt); signal (mutex) ;  
}while(1);
```

Note that, if a writer is in the critical section and n readers are waiting, then one reader is queued on `wrt`, and $n - 1$ readers are queued on `mutex`. Also observe that, when a writer executes `signal (wrt)`, we may resume the execution of either the waiting readers or a single waiting writer.

For Reference: <https://www.geeksforgeeks.org/readers-writers-problem-set-1-introduction-and-readers-preference-solution/?ref=lbp>

Lecture #23

The Dining-Philosophers Problem

Consider five philosophers who spend their lives thinking and eating. The philosophers share a common circular table surrounded by five chairs, each belonging to one philosopher. In the center of the table is a bowl of rice, and the table is laid with five single chopsticks. When a philosopher thinks, she does not interact with her colleagues. From time to time, a philosopher gets hungry and tries to pick up the two chopsticks that are closest to her (the chopsticks that are between her and her left and right neighbors). A philosopher may pick up only one chopstick at a time. Obviously, she cannot pick up a chopstick that is already in the hand of a neighbor. When a hungry philosopher has both her chopsticks at the same time, she eats without releasing her chopsticks. When she is finished eating, she puts down both of her chopsticks and starts thinking again.

The structure of philosopher i

```
do {  
wait (chopstick[i]) ; wait  
(chopstick[(i+1) % 5] ) ;  
...  
eat  
...  
signal (chopstick [i] ; signal(chopstick[(i+1)  
% 5] ) ;  
...  
think  
...  
} while (1);  
.
```

The dining-philosophers problem is considered a classic synchronization problem, neither because of its practical importance nor because computer scientists dislike philosophers, but because it is an example of a large class of concurrency-control problems. It is a simple representation of the need to allocate several resources among several processes in a deadlock- and starvation free manner.

One simple solution is to represent each chopstick by a semaphore. A philosopher tries to grab the chopstick by executing a wait operation on that semaphore; she releases her chopsticks by executing the signal operation on the appropriate semaphores. Thus, the shared data are semaphore chopstick [5] ; where all the elements of chopstick are initialized to 1.

Although this solution guarantees that no two neighbors are eating simultaneously, it nevertheless must be rejected because it has the possibility of creating a deadlock. Suppose that all five philosophers become hungry simultaneously, and each grabs her left chopstick. All the elements of chopstick will now be equal to 0. When each philosopher tries to grab her right chopstick, she will be delayed forever.

Use an asymmetric solution; that is, an odd philosopher picks up first her left chopstick and then her right chopstick, whereas an even philosopher picks up her right chopstick and then her left chopstick. Finally, any satisfactory solution to the dining-philosophers problem must guard against the possibility that one of the philosophers will starve to death.

For Reference: <https://www.geeksforgeeks.org/dining-philosopher-problem-using-semaphores/?ref=lbp>

Lecture #24

Monitors

Another high-level synchronization construct is the monitor type. A monitor is characterized by a set of programmer-defined operators. The representation of a monitor type consists of declarations of variables whose values define the state of an instance of the type, as well as the bodies of procedures or functions that implement operations on the type. The syntax of a monitor is

The representation of a monitor type cannot be used directly by the various processes. Thus, a procedure defined within a monitor can access only those variables declared locally within the monitor and its formal parameters. Similarly, the local variables of a monitor can be accessed by only the local procedures.

The monitor construct ensures that only one process at a time can be active within the monitor. Consequently, the programmer does not need to code this synchronization constraint explicitly. However, the monitor construct, as defined so far, is not sufficiently powerful for modeling some

synchronization schemes. For this purpose, we need to define additional synchronization mechanisms. These mechanisms are provided by the condition operations construct. A programmer who needs to write her own tailor-made synchronization scheme can define one or more variables of type condition: condition x , y ;

The only operations that can be invoked on a condition variable are wait and signal. The operation means that the process invoking this operation is suspended until another process invokes

The x . signal () operation resumes exactly one suspended process. If no process is suspended, then the signal operation has no effect; that is, the state of x is as though the operation were never executed (Figure 7.21). Contrast this operation with the signal operation associated with semaphores, which always affects the state of the semaphore. Now suppose that, when the x . signal () operation is invoked by a process P , there is a suspended process Q associated with condition x . Clearly, if the operations suspended process Q is allowed to resume its execution, the signaling process P must wait. Otherwise, both P and Q will be active simultaneously within the monitor. Note, however, that both processes can conceptually continue with their execution. Two possibilities exist:

1. P either waits until Q leaves the monitor, or waits for another condition.
2. Q either waits until P leaves the monitor, or waits for another condition.

There are reasonable arguments in favor of adopting either option 1 or option 2. Since P was already executing in the monitor, choice 2 seems more reasonable. However, if we allow process P to continue, the "logical" condition for which Q was waiting may no longer hold by the time Q is resumed. Choice 1 was advocated by Hoare, mainly because the preceding argument in favor of it translates directly to simpler and more elegant proof rules. A compromise between these two choices was adopted in the language Concurrent C. When process P executes the signal operation, process Q is immediately resumed. This model is less powerful than Hoare's, because a process cannot signal more than once during a single procedure call.

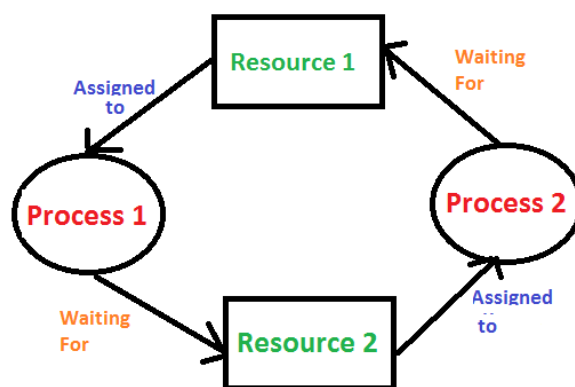
Lecture #25

Deadlocks

A set of process is in a deadlock state if each process in the set is waiting for an event that can be caused by only another process in the set. In other words, each member of the set of deadlock processes is waiting for a resource that can be released only by a deadlock process. None of the processes can run, none of them can release any resources, and none of them can be awakened.

The resources may be either physical or logical. Examples of physical resources are Printers, Hard Disc Drives, Memory Space, and CPU Cycles. Examples of logical resources are Files, Semaphores, and Monitors.

The simplest example of deadlock is where process 1 has been allocated non-shareable resources A (say a Hard Disc drive) and process 2 has been allocated non-shareable resource B (say a printer). Now, if it turns out that process 1 needs resource B (printer) to proceed and process 2 needs resource A (Hard Disc drive) to proceed and these are the only two processes in the system, each is blocked the other and all useful work in the system stops. This situation is termed deadlock. The system is in deadlock state because each process holds a resource being requested by the other process neither process is willing to release the resource it holds.



Preemptable and Nonpreemptable Resources

Resources come in two flavors: preemptable and nonpreemptable.

- ➔ A preemptable resource is one that can be taken away from the process with no ill effects. Memory is an example of a preemptable resource.
- ➔ A nonpreemptable resource is one that cannot be taken away from process (without causing ill effect). For example, CD resources are not preemptable at an arbitrary moment.
- ➔ Reallocating resources can resolve deadlocks that involve preemptable resources. Deadlocks that involve nonpreemptable resources are difficult to deal with.

Under the normal mode of operation, a process may utilize a resource in only the following sequence:

1. **Request:** If the request cannot be granted immediately, then the requesting process must wait until it can acquire the resource.
2. **Use:** The process can operate on the resource.
3. **Release:** The process releases the resource.

Necessary Conditions for Deadlock

Coffman (1971) identified four conditions that must hold simultaneously for there to be a deadlock.

1. **Mutual Exclusion Condition:** The resources involved are non-shareable.

Explanation: At least one resource must be held in a non-shareable mode, that is, only one process at a time claims exclusive control of the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.

2. Hold and Wait Condition: Requesting process hold already the resources while waiting for requested resources.

Explanation: There must exist a process that is holding a resource already allocated to it while waiting for additional resource that are currently being held by other processes.

4. No-Preemptive Condition: Resources already allocated to a process cannot be preempted.

Explanation: Resources cannot be removed from the processes are used to completion or released voluntarily by the process holding it.

4. Circular Wait Condition: The processes in the system form a circular list or chain where each process in the list is waiting for a resource held by the next process in the list. There exists a set $\{P_0, P_1, \dots, P_0\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_0 is waiting for a resource that is held by P_0 .

Note: It is not possible to have a deadlock involving only one single process. The deadlock involves a circular “hold-and-wait” condition between two or more processes, so “one” process cannot hold a resource, yet be waiting for another resource that it is holding. In addition, deadlock is not possible between two threads in a process, because it is the process that holds resources, not the thread that is, each thread has access to the resources held by the process.

Resource-Allocation Graph

Deadlocks can be described in terms of a directed graph called a **system resource-allocation graph**.

This graph consists of a set of vertices V and a set of edges E . The set of vertices V is partitioned into two different types of nodes $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the active processes in the system, and $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.

A directed edge from process P_i to resource type R_j is denoted by $P_i \rightarrow R_j$; it signifies that process P_i requested an instance of resource type R_j and is currently waiting for that resource. A directed edge $P_i \rightarrow R_j$ is called a request edge.

A directed edge from resource type R_j to process P_i is denoted by $R_j \rightarrow P_i$; it signifies that an instance of resource type R_j has been allocated to process P_i . A directed edge $R_j \rightarrow P_i$ is called an assignment edge.

Pictorially, we represent each process P_i as a circle and each resource type R_j as a square. Since resource type R_j may have more than one instance, we represent each such instance as a dot within the square. A request edge points to only the square R_j , whereas an assignment edge must designate one of the dots in the square.



The resource-allocation graph shown below depicts the following situation. The sets

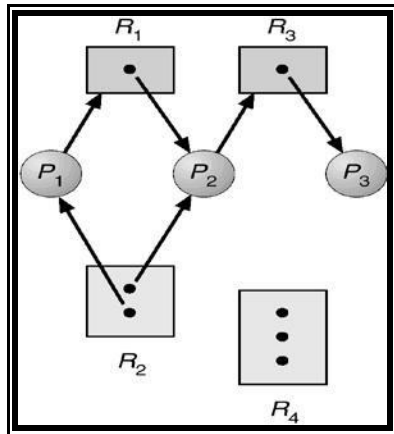
P , R , and E :

$P = \{P_1, P_2, P_3\}$

$R = \{R_1, R_2, R_3, R_4\}$

$E = \{P_1 \rightarrow R_1, P_2 \rightarrow R_3, R_1 \rightarrow P_2, R_2 \rightarrow P_2, R_2 \rightarrow P_1, R_3 \rightarrow P_3\}$ Resource instances:

- One instance of resource type R_1
- Two instances of resource type R_2
- One instance of resource type R_3
- Three instances of resource type R_4 Process states:
- Process P_1 is holding an instance of resource type R_2 , and is waiting for an instance of resource type R_1 .
- Process P_2 is holding an instance of R_1 and R_2 , and is waiting for an instance of resource type R_3 .
- Process P_3 is holding an instance of R_3 .



Example of a Resource Allocation Graph

Given the definition of a resource-allocation graph, it can be shown that, if the graph contains no cycles, then no process in the system is deadlocked. If the graph does contain a cycle, then a deadlock may exist.

If each resource type has exactly one instance, then a cycle implies that a deadlock has occurred. If the cycle involves only a set of resource types, each of which has only a single instance, then a deadlock has occurred. Each process involved in the cycle is deadlocked. In this case, a cycle in the graph is both a necessary and a sufficient condition for the existence of deadlock.

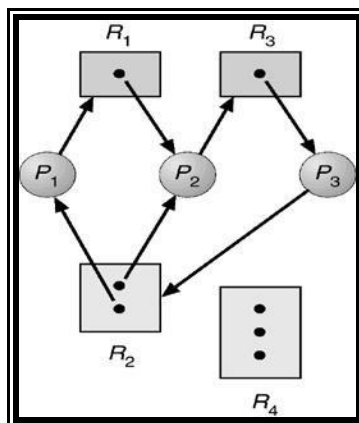
If each resource type has several instances, then a cycle does not necessarily imply that a deadlock has occurred. In this case, a cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock.

To illustrate this concept, let us return to the resource-allocation graph depicted in Figure. Suppose that process P3 requests an instance of resource type R2. Since no resource instance is currently available, a request edge $P3 \rightarrow R2$ is added to the graph. At this point, two minimal cycles exist in the system:

$P1 \rightarrow R1 \rightarrow P2 \rightarrow R3 \rightarrow P3 \rightarrow R2 \rightarrow P1$

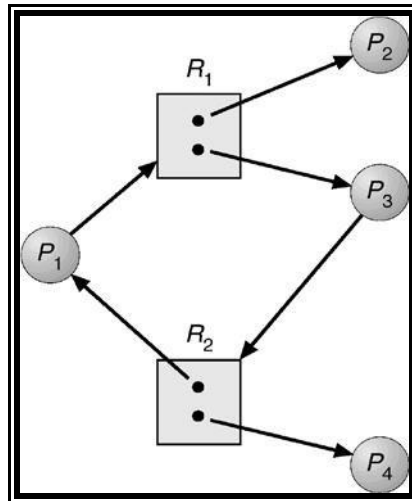
$P2 \rightarrow R3 \rightarrow P3 \rightarrow R2 \rightarrow P2$

Processes P1, P2, and P3 are deadlocked. Process P2 is waiting for the resource R3, which is held by process P3. Process P3, on the other hand, is waiting for either process P1 or process P2 to release resource R2. In addition, process P1 is waiting for process P2 to release resource R1.



Resource Allocation Graph with a deadlock

Now consider the resource-allocation graph in the following Figure. In this example, we also have a cycle. However, there is no deadlock. Observe that process P4 may release its instance of resource type R2. That resource can then be allocated to P3, breaking the cycle.



Resource Allocation Graph with a Cycle but No Deadlock

METHODS OF HANDLING DEADLOCK

In general, there are four strategies of dealing with deadlock problem:

1. **Deadlock Prevention:** Prevent deadlock by resource scheduling so as to negate at least one of the four conditions.
2. **Deadlock Avoidance:** Avoid deadlock by careful resource scheduling.
3. **Deadlock Detection and Recovery:** Detect deadlock and when it occurs, take steps to recover.
4. **The Ostrich Approach:** Just ignore the deadlock problem altogether.

DEADLOCK PREVENTION

A deadlock may be prevented by denying any one of the conditions.

- **Elimination of “Mutual Exclusion” Condition:** The mutual exclusion condition must hold for nonsharable resources. That is, several processes cannot simultaneously share a single resource. This condition is difficult to eliminate because some resources, such as the Hard disc drive and printer, are inherently non-shareable. Note that shareable resources like read-only-file do not require mutually exclusive access and thus cannot be involved in deadlock.
- **Elimination of “Hold and Wait” Condition:** There are two possibilities for elimination of the second condition. The first alternative is that a process request be granted all of the resources it needs at once, prior to execution. The second alternative is to disallow a process from requesting resources whenever it has previously allocated resources. This strategy requires that all of the resources a process will need must be requested at once. The system must grant resources on “all or none” basis. If the complete set of resources needed by a process is not currently available, then the process must wait until the complete set is available. While the process waits, however, it may not hold any resources. Thus the “wait for” condition is denied and deadlocks cannot occur. This strategy can lead to serious waste of resources.
- **Elimination of “No-preemption” Condition:** The non-preemption condition can be alleviated by forcing a process waiting for a resource that cannot immediately be allocated to relinquish all of its currently held resources, so that other processes may use them to finish. This strategy requires that when a process that is holding some resources is denied a request for additional resources. The process must release its held resources and, if necessary, request them again together with additional resources. Implementation of this strategy denies the “no-preemptive” condition effectively.
- **Elimination of “Circular Wait” Condition:** The last condition, the circular wait, can be denied by imposing a total ordering on all of the resource types and then forcing, all processes to request the resources in order (increasing or decreasing). This strategy impose a total ordering of all resources types, and to require that each process requests resources in a numerical order (increasing or decreasing) of enumeration. With this rule, the resource allocation graph can never have a cycle.

For example, provide a global numbering of all the resources, as shown

- 1 $\equiv$ Card reader
- 2 $\equiv$ Printer
- 3 $\equiv$ Optical driver
- 4 $\equiv$ HDD
- 5 $\equiv$ Card punch

Now the rule is this: processes can request resources whenever they want to, but all requests must be made in numerical order. A process may request first printer and then a HDD (order: 2, 4), but it may not request first an optical driver and then a printer (order: 3, 2). The problem with this strategy is that it may be impossible to find an ordering that satisfies everyone.

DEADLOCK AVOIDANCE

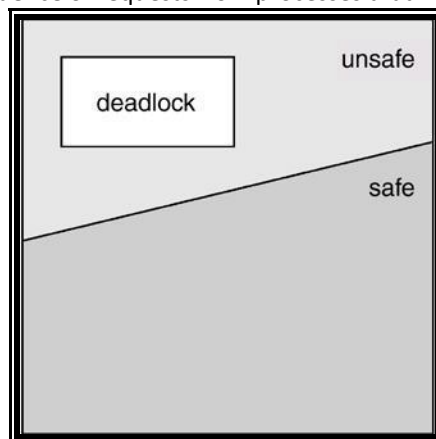
This approach to the deadlock problem anticipates deadlock before it actually occurs. This approach employs an algorithm to assess the possibility that deadlock could occur and act accordingly. If the necessary conditions for a deadlock are in place, it is still possible to avoid deadlock by being careful when resources are allocated. It employs the most famous deadlock avoidance algorithm that is the Banker's algorithm.

A deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that a circular wait condition can never exist. The resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes.

Safe and Unsafe States

A system is said to be in a **Safe State**, if there is a safe execution sequence. An execution sequence is an ordering for process execution such that each process runs until it terminates or blocked and all request for resources are immediately granted if the resource is available.

A system is said to be in an **Unsafe State**, if there is no safe execution sequence. An unsafe state may not be deadlocked, but there is at least one sequence of requests from processes that would make the system deadlocked.



(Relation between Safe, Unsafe and Deadlocked States)

Resource-Allocation Graph Algorithm

The deadlock avoidance algorithm uses a variant of the resource-allocation graph to avoid a deadlocked state. It introduces a new type of edge, called a **claim edge**. A claim edge $P_i \rightarrow R_j$ indicates that process P_i may request resource R_j at some time in the future. This edge resembles a request edge in direction, but is represented by a dashed line. When process P_i requests resource R_j , the claim edge $P_i \rightarrow R_j$ is converted to a request edge. Similarly, when a resource R_j is released by P_i , the assignment edge $R_j \rightarrow P_i$ is reconverted to a claim edge $P_i \rightarrow R_j$.

Suppose that process P_i requests resource R_j . The request can be granted only if converting the request edge $P_i \rightarrow R_j$ to an assignment edge $R_j \rightarrow P_i$ that does not result in the formation of a cycle in the resource-allocation graph. An algorithm for detecting a cycle in this graph is called cycle detection algorithm.

If no cycle exists, then the allocation of the resource will leave the system in a safe state. If a cycle is found, then the allocation will put the system in an unsafe state. Therefore, process P_i will have to wait for its requests to be satisfied.

Banker's algorithm

The **Banker's algorithm** is a resource allocation & deadlock avoidance algorithm developed by Edsger Dijkstra that test for safety by simulating the allocation of pre-determined maximum possible amounts of all resources. Then it makes a "safe-state" check to test for possible deadlock conditions for all other pending activities, before deciding whether allocation should be allowed to continue.

The Banker's algorithm is run by the operating system whenever a process requests resources. The algorithm prevents deadlock by denying or postponing the request if it determines that accepting the request could put the system in an unsafe state (one where deadlock could occur).

For the Banker's algorithm to work, it needs to know three things:

- How much of each resource could possibly request by each process.
- How much of each resource is currently holding by each process.
- How much of each resource the system currently has available.

Resources may be allocated to a process only if it satisfies the following conditions:

1. $\text{request} \leq \text{max}$, else set error as process has crossed maximum claim made by it.
2. $\text{request} \leq \text{available}$, else process waits until resources are available.

Several data structures must be maintained to implement the banker's algorithm. These data structures encode the state of the resource-allocation system. Let n be the number of processes in the system and m be the number of resource types. We need the following data structures:

Available: A vector of length m indicates the number of available resources of each type. If $\text{Available}[j] = k$, there are k instances of resource type R_j available.

Max: An $n \times m$ matrix defines the maximum demand of each process. If $\text{Max}[i,j] = k$, then process P_i may request at most k instances of resource type R_j .

Allocation: An $n \times m$ matrix defines the number of resources of each type currently allocated to each process. If $\text{Allocation}[i,j] = k$, then process P_i is currently allocated k instances of resource type R_j .

Need: An $n \times m$ matrix indicates the remaining resource need of each process. If $\text{Need}[i,j] = k$, then process P_i may need k more instances of resource type R_j to complete its task. Note that $\text{Need}[i,j] = \text{Max}[i,j] - \text{Allocation}[i,j]$.

These data structures vary over time in both size and value. The vector Allocation_i specifies the resources currently allocated to process P_i ; the vector Need_i specifies the additional resources that process P_i may still request to complete its task.

Safety Algorithm

The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1. Let Work and Finish be vectors of length m and n , respectively. Initialize $\text{Work} := \text{Available}$ and $\text{Finish}[i] := \text{false}$ for $i = 1, 2, \dots, n$.
2. Find an i such that
 - a. $\text{Finish}[i] = \text{false}$
 - b. $\text{Need}_i \leq \text{Work}$.

If no such i exists, go to step 4.

3. $\text{Work} := \text{Work} + \text{Allocation}_i$ $\text{Finish}[i] := \text{true}$ go to step 2.

4. If $\text{Finish}[i] = \text{true}$ for all i , then the system is in a safe state.

This algorithm may require an order of $m \times n^2$ operations to decide whether a state is safe.

Resource-Request Algorithm

Let $Request_i$ be the request array for process P_i . $Request_i[j] = k$ means process P_i wants k instances of resource type R_j . When a request for resources is made by process P_i , the following actions are taken:

1) If $Request_i \leq Need_i$

Goto step (2); otherwise, raise an error condition, since the process has exceeded its maximum claim.

2) If $Request_i \leq Available$

Goto step (3); otherwise, P_i must wait, since the resources are not available.

3) Have the system pretend to have allocated the requested resources to process P_i by modifying the state as follows:

$Available = Available - Request_i$

$Allocation_i = Allocation_i + Request_i$

$Need_i = Need_i - Request_i$

If the resulting resource-allocation state is safe, the transaction is completed and process P_i is allocated its resources. However, if the new state is unsafe, then P_i must wait for $Request_i$ and the old resource-allocation state is restored.

Consider a system with five processes P_0 through P_4 and three resource types A,B,C. Resource type A has 10 instances, resource type B has 5 instances, and resource type C has 7 instances. Suppose that, at time T_0 , the following

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			

snapshot of the system has been taken:

Q.1: What will be the content of the Need matrix?

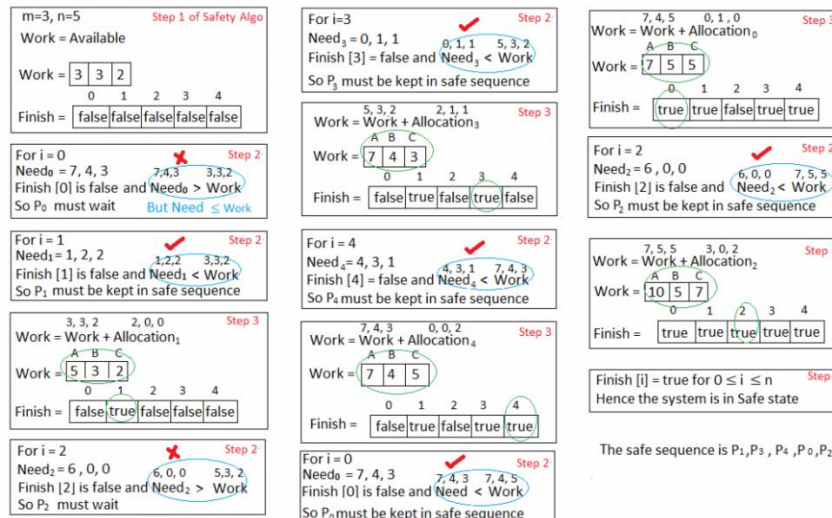
$Need[i, j] = Max[i, j] - Allocation[i, j]$

So, the content of Need Matrix is:

Process	Need		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Q.2: Is the system in a safe state? If Yes, then what is the safe sequence?

Applying the Safety algorithm on the given system,



Q.3: What will happen if process P1 requests one additional instance of resource type A and two instances of resource type C?

Request₁ = 1, 0, 2

To decide whether the request is granted we use Resource Request algorithm

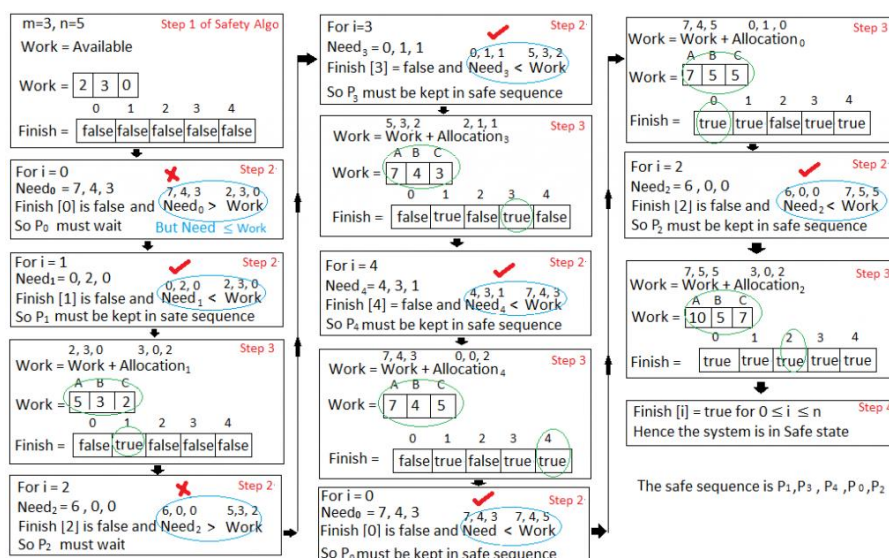
Step 1
 $1, 0, 2 < 7, 4, 3$ ✓

Step 2
 $1, 0, 2 < 3, 3, 2$ ✓

Step 3
 Available = Available - Request₁
 Allocation₁ = Allocation₁ + Request₁
 Need₁ = Need₁ - Request₁

Process	Allocation	Need	Available
	A B C	A B C	A B C
P ₀	0 1 0	7 4 3	2 3 0
P ₁	3 0 2	0 2 0	
P ₂	3 0 2	6 0 0	
P ₃	2 1 1	0 1 1	
P ₄	0 0 2	4 3 1	

We must determine whether this new system state is safe. To do so, we again execute Safety algorithm on the above data structures.



We claim that the system is currently in a safe state. Indeed, the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies the safety criteria. Suppose now that process P1 requests one additional instance of resource type A and two instances of resource type C,

so Request₁ = (1,0,2). To decide whether this request can be immediately granted, we first check that Request₁ ≤ Available (that is, (1,0,2) ≤ (3,3,2)), which is true. We then pretend that this request has been fulfilled, and we arrive at the following new state:

We must determine whether this new system state is safe. To do so, we execute our safety algorithm and find that the sequence <P₁, P₃, P₄, P₀, P₂> satisfies our safety requirement. Hence, we can immediately grant the request of process P₁.

However, that when the system is in this state, a request for (3,3,0) by P₄ cannot be granted, since the resources are not available. A request for (0,2,0) by P₀ cannot be granted, even though the resources are available, since the resulting state is unsafe.

DEADLOCK DETECTION

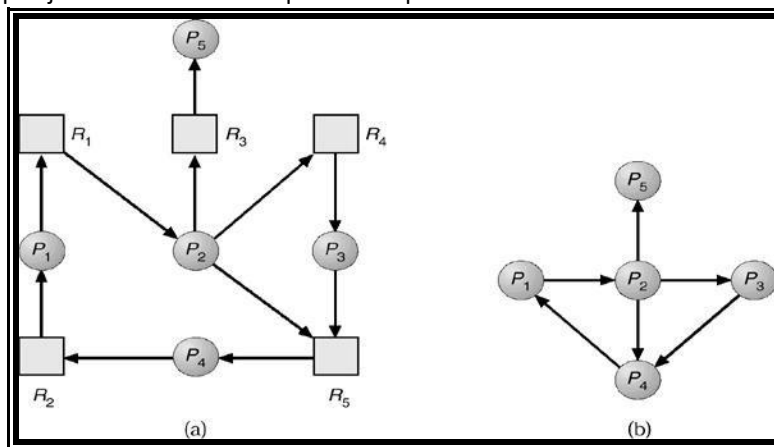
If a system does not employ either a deadlock-prevention or a deadlock avoidance algorithm, then a deadlock situation may occur. In this environment, the system must provide:

- An algorithm that examines the state of the system to determine whether a deadlock has occurred.
- An algorithm to recover from the deadlock.

According to number of instances in each resource type, the Deadlock Detection algorithm can be classified into two categories as follows:

1. Single Instance of Each Resource Type: If all resources have only a single instance, then it can define a deadlock detection algorithm that uses a variant of the resource-allocation graph (is called a *wait-for* graph). A wait-for graph can be draw by removing the nodes of type resource and collapsing the appropriate edges from the resource-allocation graph.

An edge from P_i to P_j in a wait-for graph implies that process P_i is waiting for process P_j to release a resource that P_i needs. An edge P_i → P_j exists in a wait-for graph if and only if the corresponding resource allocation graph contains two edges P_i → R_q and R_q → P_j for some resource R_q. For Example:



((a) Resource-allocation graph. (b) Corresponding wait-for graph)

A deadlock exists in the system if and only if the wait-for graph contains a cycle. To detect deadlocks, the system needs to maintain the wait-for graph and periodically to invoke an algorithm that searches for a cycle in the graph. An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph.

2. Several Instances of a Resource Type: The following deadlock-detection algorithm is applicable to several instance of a resource type. The algorithm employs several time-varying data structures:

Available: A vector of length m indicates the number of available resources of each type.

Allocation: An $n \times m$ matrix defines the number of resources of each type currently allocated to each process. **Request:** An $n \times m$ matrix indicates the current request of each process. If Request_[i,j] = k , then process P_i is requesting k more instances of resource type R_j.

The detection algorithm is described as follows:

1. Let *Work* and *Finish* be vectors of length *m* and *n*, respectively.
Initialize, *Work* := *Available*. For *i* = 1, 2, ..., *n*,
if *Allocation_i* != 0, then *Finish[i]* := *false*; otherwise, *Finish[i]* := *true*.
 2. Find an index *i* such that both
 - a. *Finish[i]* = *false*.
 - b. *Request_i* ≤ *Work*.If no such *i* exists, go to step 4.
 3. *Work* := *Work* + *Allocation_i*
Finish[i] := *true*
go to step 2.
 4. If *Finish[i]* = *false*, for some *i*, $1 \leq i \leq n$, then the system is in a deadlock state.
if *Finish[i]* = *false*, then process *P_i* is deadlocked.
- This algorithm requires an order of $m \times n^2$ operations to detect whether the system is in a deadlocked state.

RECOVERY FROM DEADLOCK

When a detection algorithm determines that a deadlock exists, then the system or operator is responsible for handling deadlock problem. There are two options for breaking a deadlock.

1. Process Termination
2. Resource preemption
- 3.

Process Termination

There are two method to eliminate deadlocks by terminating a process as follows:

1. **Abort all deadlocked processes:** This method will break the deadlock cycle clearly by terminating all process. This method is cost effective. And it removes the partial computations completed by the processes.
2. **Abort one process at a time until the deadlock cycle is eliminated:** This method terminates one process at a time, and invokes a deadlock-detection algorithm to determine whether any processes are still deadlocked.

Resource Preemption

In resource preemption, the operator or system preempts some resources from processes and give these resources to other processes until the deadlock cycle is broken.

If preemption is required to deal with deadlocks, then three issues need to be addressed:

1. **Selecting a victim:** The system or operator selects which resources and which processes are to be preempted based on cost factor.
2. **Rollback:** The system or operator must roll back the process to some safe state and restart it from that state.
3. **Starvation:** The system or operator should ensure that resources will not always be preempted from the same process?

Unit 4

MEMORY MANAGEMENT

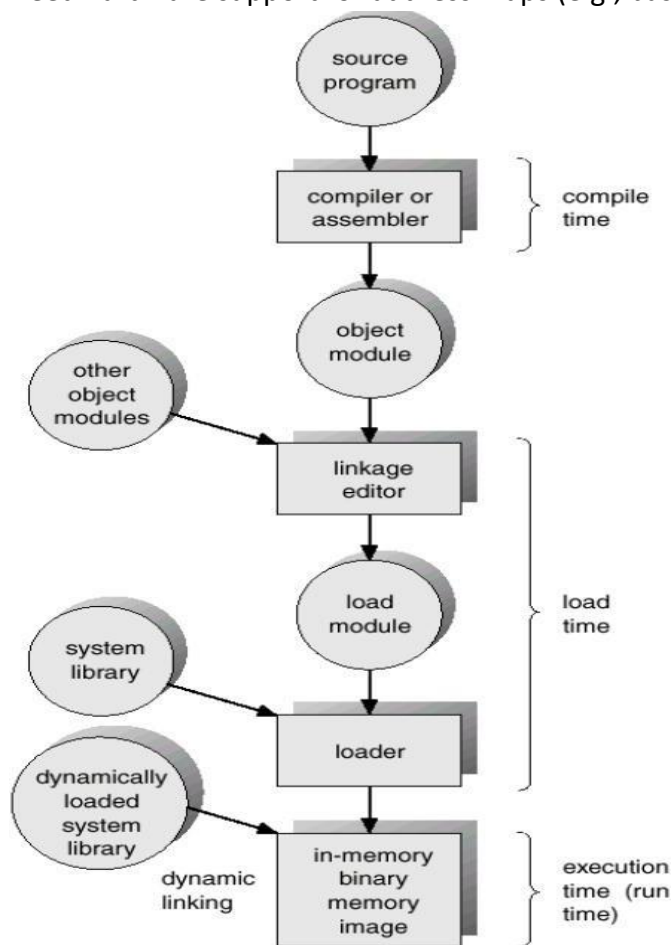
In a uni-programming system, main memory is divided into two parts: one part for the operating system (resident monitor, kernel) and one part for the user program currently being executed.

In a multiprogramming system, the “user” part of memory must be further subdivided to accommodate multiple processes. The task of subdivision is carried out dynamically by the operating system and is known as **memory management**.

Binding of Instructions and Data to Memory

Address binding of instructions and data to memory addresses can happen at three different stages.

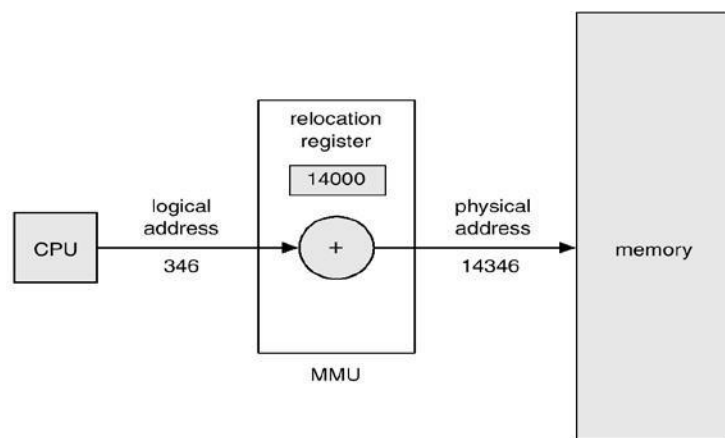
1. **Compile time:** The compile time is the time taken to compile the program or source code. During compilation, if memory location known a priori, then it generates absolute codes.
2. **Load time:** It is the time taken to link all related program file and load into the main memory. It must generate relocatable code if memory location is not known at compile time.
3. **Execution time:** It is the time taken to execute the program in main memory by processor. Binding delayed until run time if the process can be moved during its execution from one memory segment to another. Need hardware support for address maps (e.g., base and limit registers).



(Multistep processing of a user program.)

Logical- Versus Physical-Address Space

- ⇒ An address generated by the CPU is commonly referred to as a **logical address** or a **virtual address** whereas an address seen by the main memory unit is commonly referred to as a **physical address**.
- ⇒ The set of all logical addresses generated by a program is a **logical-address space** whereas the set of all physical addresses corresponding to these logical addresses is a **physical address space**.
- ⇒ Logical and physical addresses are the same in compile-time and load-time addressbinding schemes; logical (virtual) and physical addresses differ in execution-time addressbinding scheme.
- ⇒ The Memory Management Unit is a hardware device that maps virtual to physical address. In MMU scheme, the value in the relocation register is added to every address generated by a user process at the time it is sent to memory as follows:



(Dynamic relocation using a relocation register)

Dynamic Loading

- ⇒ It loads the program and data dynamically into physical memory to obtain better memory-space utilization.
- ⇒ With dynamic loading, a routine is not loaded until it is called.
- ⇒ The advantage of dynamic loading is that an unused routine is never loaded.
- ⇒ This method is useful when large amounts of code are needed to handle infrequently occurring cases, such as error routines. ⇒ Dynamic loading does not require special support from the operating system.

Dynamic Linking

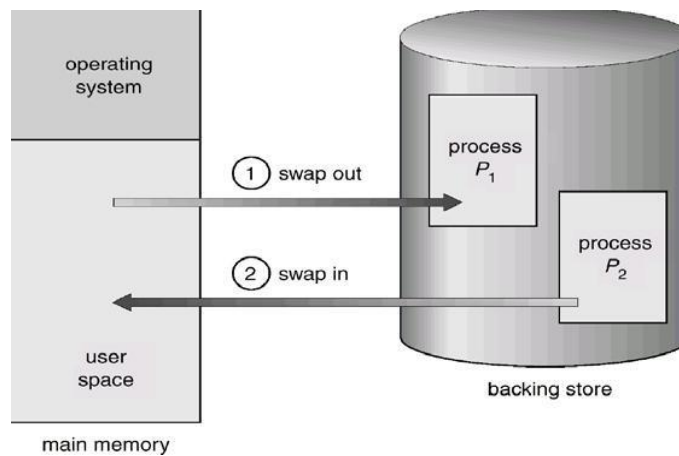
- ⇒ Linking postponed until execution time.
 - ⇒ Small piece of code (stub) used to locate the appropriate memory-resident library routine.
 - ⇒ Stub replaces itself with the address of the routine and executes the routine.
 - ⇒ Operating system needed to check if routine is in processes memory address. ⇒
- Dynamic linking is particularly useful for libraries.

Overlays

- ⇒ Keep in memory only those instructions and data that are needed at any given time.
- ⇒ Needed when process is larger than amount of memory allocated to it.
- ⇒ Implemented by user, no special support needed from operating system, programming design of overlay structure is complex.

Swapping

- ⇒ A process can be swapped temporarily out of memory to a backing store (large disc), and then brought back into memory for continued execution.
- ⇒ **Roll out, roll in:** A variant of this swapping policy is used for priority-based scheduling algorithms. If a higher-priority process arrives and wants service, the memory manager can swap out the lower-priority process so that it can load and execute the higher-priority process. When the higher-priority process finishes, the lower-priority process can be swapped back in and continued. This variant of swapping is called **roll out, roll in**.
- ⇒ Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped. ⇒ Modified versions of swapping are found on many systems (UNIX, Linux, and Windows).



(Schematic View of Swapping)

MEMORY ALLOCATION

The main memory must accommodate both the operating system and the various user processes. We need to allocate different parts of the main memory in the most efficient way possible.

The main memory is usually divided into two partitions: one for the resident operating system, and one for the user processes. We may place the operating system in either low memory or high memory. The major factor affecting this decision is the location of the interrupt vector. Since the interrupt vector is often in low memory, programmers usually place the operating system in low memory as well.

There are following two ways to allocate memory for user processes:

1. Contiguous memory allocation

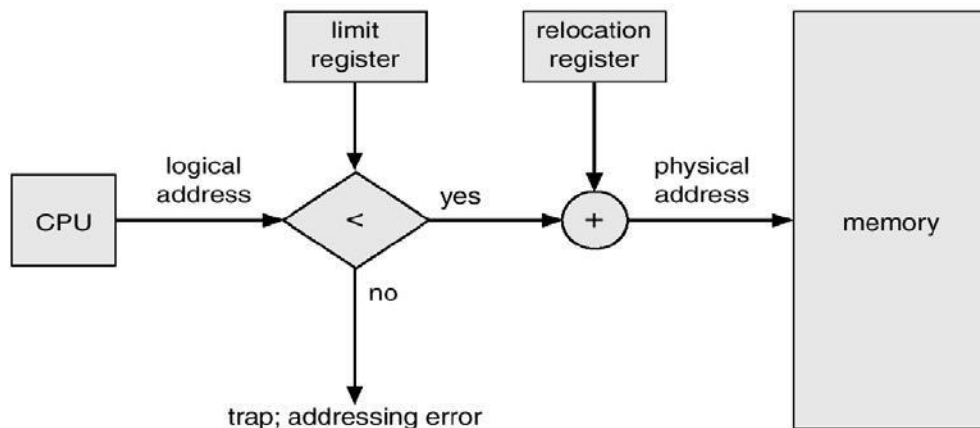
2. Non contiguous memory allocation

1. Contiguous Memory Allocation

Here, all the processes are stored in contiguous memory locations. To load multiple processes into memory, the Operating System must divide memory into multiple partitions for those processes.

Hardware Support: The relocation-register scheme used to protect user processes from each other, and from changing operating system code and data. Relocation register contains value of smallest physical address of a partition and limit register contains range of that partition.

Each logical address must be less than the limit register.



(Hardware support for relocation and limit registers)

According to size of partitions, the multiple partition schemes are divided into two types:

- i. Multiple fixed partition/ multiprogramming with fixed task(MFT)
- ii. Multiple variable partition/ multiprogramming with variable task(MVT)

i. Multiple fixed partitions: Main memory is divided into a number of static partitions at system generation time. In this case, any process whose size is less than or equal to the partition size can be loaded into any available partition. If all partitions are full and no process is in the Ready or Running state, the operating system can swap a process out of any of the partitions and load in another process, so that there is some work for the processor.

Advantages: Simple to implement and little operating system overhead.

Disadvantage: * Inefficient use of memory due to internal fragmentation.

* Maximum number of active processes is fixed.

ii. Multiple variable partitions: With this partitioning, the partitions are of variable length and number. When a process is brought into main memory, it is allocated exactly as much memory as it requires and no more.

Advantages: No internal fragmentation and more efficient use of main memory.

Disadvantages: Inefficient use of processor due to the need for compaction to counter external fragmentation. **Partition Selection policy:**

When the multiple memory holes (partitions) are large enough to contain a process, the operating system must use an algorithm to select in which hole the process will be loaded. The partition selection algorithm are as follows:

⇒ **First-fit:** The OS looks at all sections of free memory. The process is allocated to the first hole found that is big enough size than the size of process.

⇒ **Next Fit:** The next fit search starts at the last hole allocated and The process is allocated to the next hole found that is big enough size than the size of process.

⇒ **Best-fit:** The Best Fit searches the entire list of holes to find the smallest hole that is big enough size than the size of process.

⇒ **Worst-fit:** The Worst Fit searches the entire list of holes to find the largest hole that is big enough size than the size of process.

Fragmentation: The wasting of memory space is called fragmentation. There are two types of fragmentation as follows:

1. **External Fragmentation:** The total memory space exists to satisfy a request, but it is not contiguous. This wasted space not allocated to any partition is called external fragmentation. The external fragmentation can be reduce by compaction. The goal is to shuffle the memory contents to place all free memory together in one large block. Compaction is possible only if relocation is dynamic, and is done at execution time.
2. **Internal Fragmentation:** The allocated memory may be slightly larger than requested memory. The wasted space within a partition is called internal fragmentation. One method to reduce internal fragmentation is to use partitions of different size.

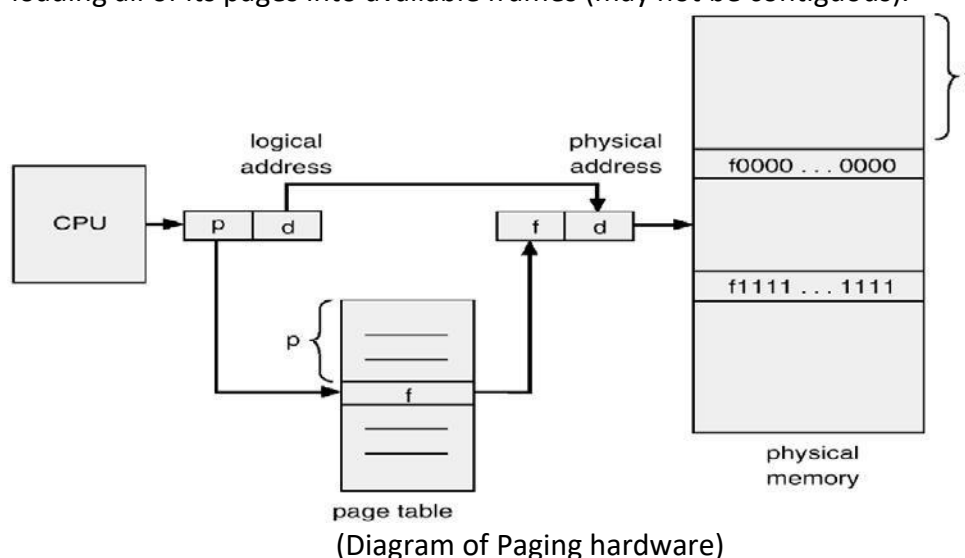
2. Noncontiguous memory allocation

In noncontiguous memory allocation, it is allowed to store the processes in non contiguous memory locations. There are different techniques used to load processes into memory, as follows:

1. Paging
2. Segmentation
3. Virtual memory paging(Demand paging) etc.

PAGING

Main memory is divided into a number of equal-size blocks, are called **frames**. Each process is divided into a number of equal-size block of the same length as frames, are called **Pages**. A process is loaded by loading all of its pages into available frames (may not be contiguous).



Process of Translation from logical to physical addresses

⇒ Every address generated by the CPU is divided into two parts: a page number (**p**) and a page offset (**d**). The page number is used as an index into a page table.

⇒ The page table contains the base address of each page in physical memory. This base address is combined with the page offset to define the physical memory address that is sent to the memory unit.

⇒ If the size of logical-address space is 2^m and a page size is 2^n addressing units (bytes or words), then the high-order $(m - n)$ bits of a logical address designate the page number and the n low-order bits designate the page offset. Thus, the logical address is as follows:

page number	page offset
p	d
$m - n$	n

Where p is an index into the page table and d is the displacement within the page.

Example:

Consider a page size of 4 bytes and a physical memory of 32 bytes (8 pages), we show how the user's view of memory can be mapped into physical memory. Logical address 0 is page 0, offset 0. Indexing into the page table, we find that page 0 is in frame 5. Thus, logical address 0 maps to physical address 20 ($= (5 \times 4) + 0$). Logical address 3 (page 0, offset 3) maps to physical address 23 ($= (5 \times 4) + 3$). Logical address 4 is page 1, offset 0; according to the page table, page 1 is mapped to frame 6. Thus, logical address 4 maps to physical address 24 ($= (6 \times 4) + 0$). Logical address 13 maps to physical address 9 ($= (2 \times 4) + 1$).

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

Hardware Support for Paging:

Each operating system has its own methods for storing page tables. Most operating systems allocate a page table for each process. A pointer to the page table is stored with the other register values (like the instruction counter) in the process control block. When the dispatcher is told to start a process, it must reload the user registers and define the correct hardware page table values from the stored user page table. **Implementation of Page Table**

⇒ Generally, Page table is kept in main memory. The Page Table Base Register (PTBR) points to the page table. And Page-table length register (PRLR) indicates size of the page table.

⇒ In this scheme every data/instruction access requires two memory accesses. One for the page table and one for the data/instruction.

⇒ The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**.

Paging Hardware With TLB

The TLB is an associative and high-speed memory. Each entry in the TLB consists of two parts: a key (or tag) and a value. The TLB is used with page tables in the following way.

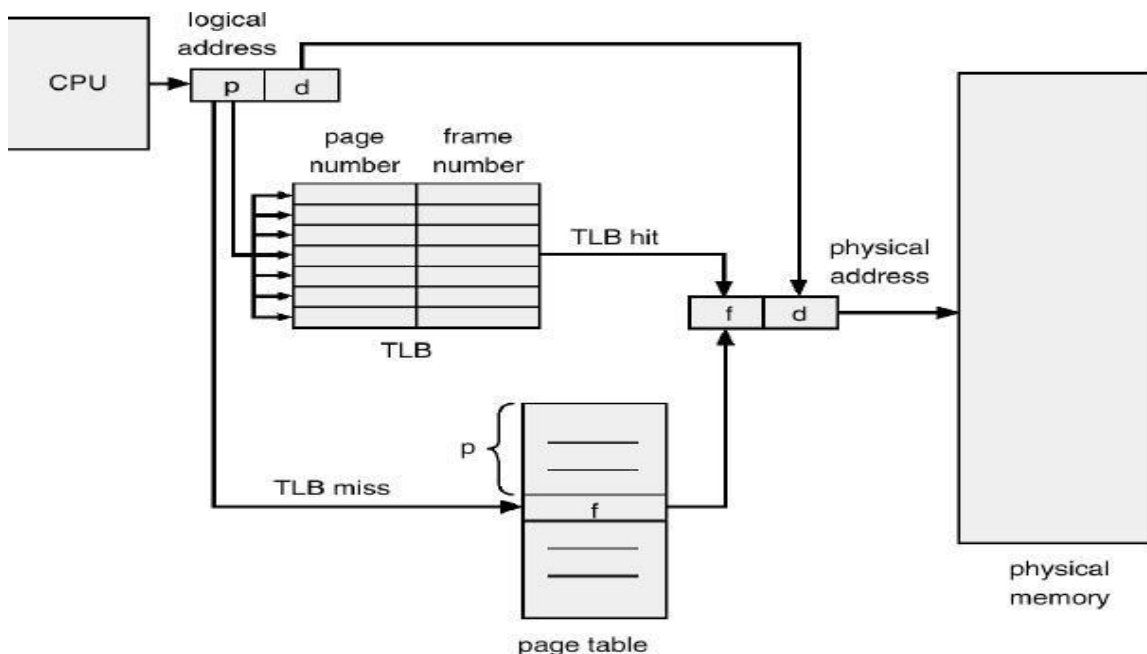
⇒ The TLB contains only a few of the page-table entries. When a logical address is generated by the CPU, its page number is presented to the TLB.

⇒ If the page number is found (known as a **TLB Hit**), its frame number is immediately available and is used to access memory. It takes only one memory access.

⇒ If the page number is not in the TLB (known as a **TLB miss**), a memory reference to the page table must be made. When the frame number is obtained, we can use it to access memory. It takes two memory accesses.

⇒ In addition, it stores the page number and frame number to the TLB, so that they will be found quickly on the next reference.

⇒ If the TLB is already full of entries, the operating system must select one for replacement by using replacement algorithm.



(Paging hardware with TLB)

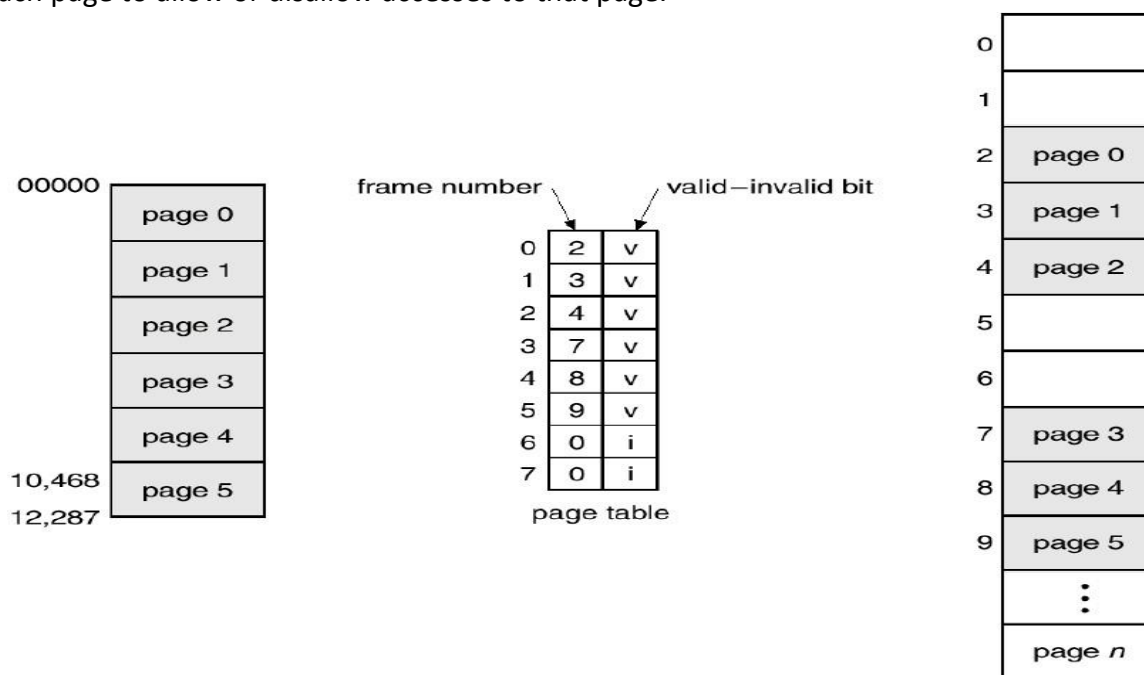
The percentage of times that a particular page number is found in the TLB is called the **hit ratio**. The effective access time (EAT) is obtained as follows:

$$EAT = HR \times (TLBAT + MAT) + MR \times (TLBAT + 2 \times MAT)$$

Where HR: Hit Ratio, TLBAT: TLB access time, MAT: Memory access time, MR: Miss Ratio.

Memory protection in Paged Environment:

- ⇒ Memory protection in a paged environment is accomplished by protection bits that are associated with each frame. These bits are kept in the page table.
- ⇒ One bit can define a page to be read-write or read-only. This protection bit can be checked to verify that no writes are being made to a read-only page. An attempt to write to a read-only page causes a hardware trap to the operating system (or memory-protection violation).
- ⇒ One more bit is attached to each entry in the page table: a **valid-invalid** bit. When this bit is set to "valid," this value indicates that the associated page is in the process' logical address space, and is a legal (or valid) page. If the bit is set to "invalid," this value indicates that the page is not in the process' logical-address space.
- ⇒ Illegal addresses are trapped by using the valid-invalid bit. The operating system sets this bit for each page to allow or disallow accesses to that page.



(Valid (v) or invalid (i) bit in a page table)

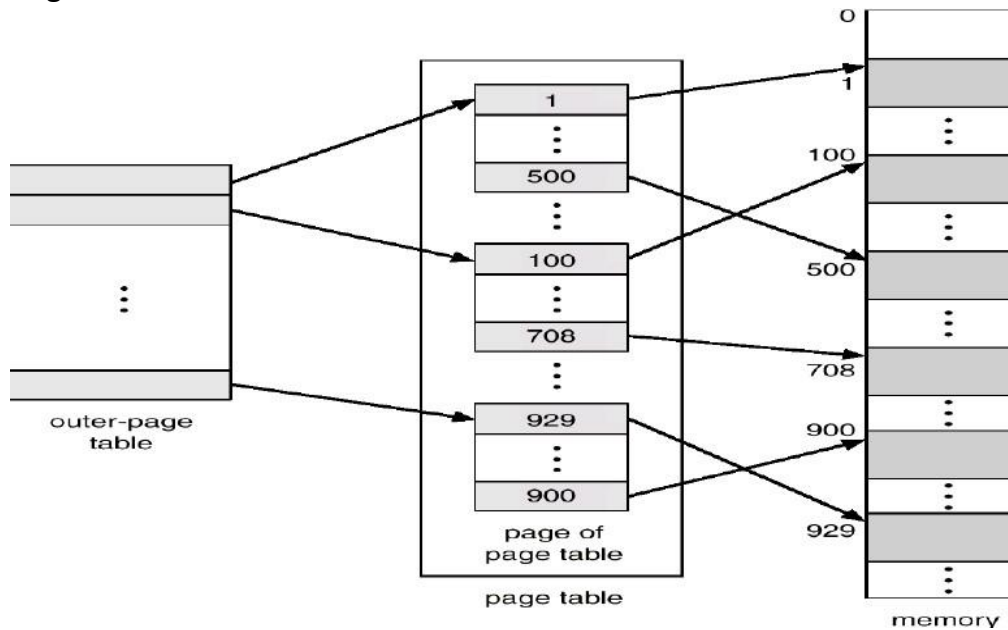
There are different structures of page table described as follows:

- 1. Hierarchical Page table:** When the number of pages is very high, then the page table takes large amount of memory space. In such cases, we use multilevel paging scheme for reducing size of page table. A simple technique is a two-level page table. Since the page table is paged, the page number is further divided into parts: page number and page offset. Thus, a logical address is as follows:

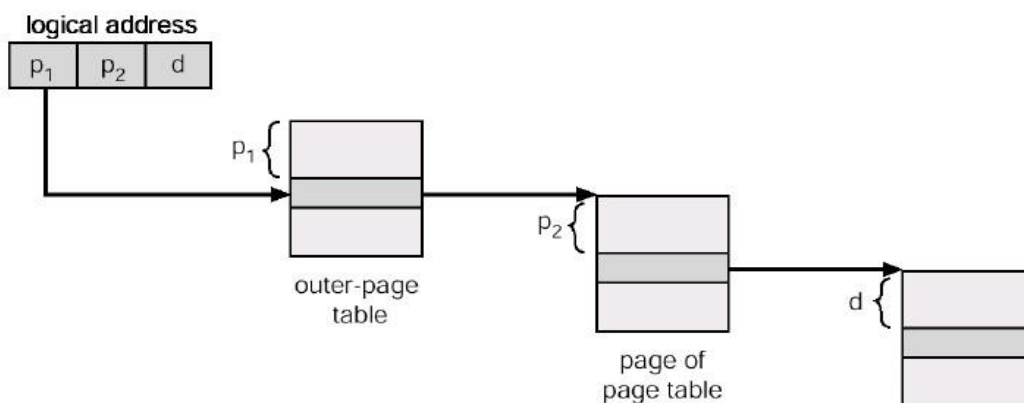
page number		page offset
p_1	p_2	d

Where p_i is an index into the outer page table, and p_2 is the displacement within the page of the outer page table.

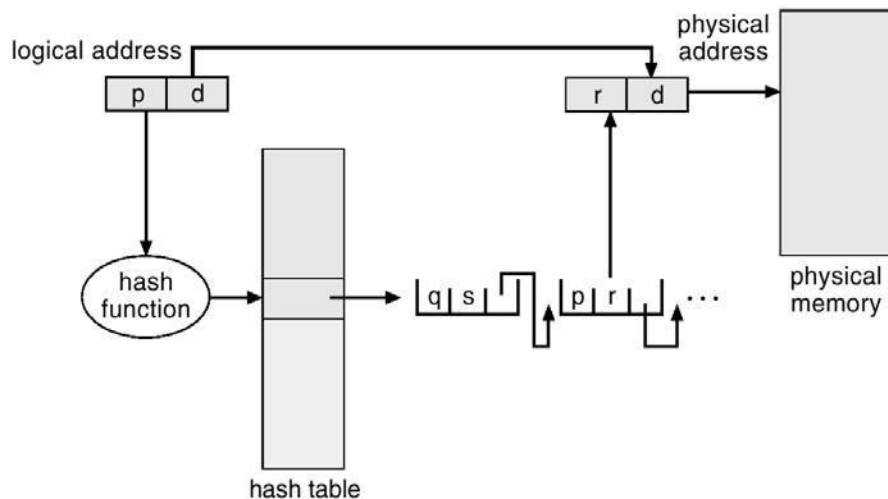
Two-Level Page-Table Scheme:



Address translation scheme for a two-level paging architecture:

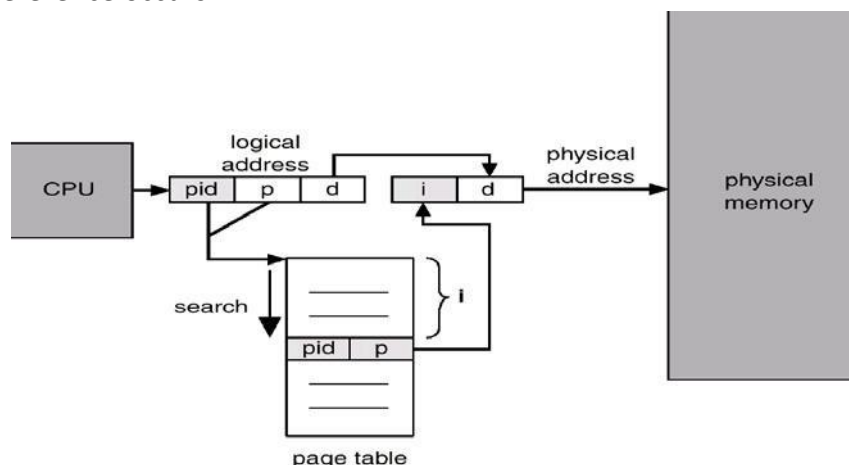


2. Hashed Page Tables: This scheme is applicable for address space larger than 32bits. In this scheme, the virtual page number is hashed into a page table. This page table contains a chain of elements hashing to the same location. Virtual page numbers are compared in this chain searching for a match. If a match is found, the corresponding physical frame is extracted.



3. Inverted Page Table:

- ⇒ One entry for each real page of memory.
- ⇒ Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page.
- ⇒ Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs.



Shared Pages

Shared code

- ⇒ One copy of read-only (reentrant) code shared among processes (i.e., text editors, compilers, window systems).
- ⇒ Shared code must appear in same location in the logical address space of all processes.

Private code and data

- ⇒ Each process keeps a separate copy of the code and data.
- ⇒ The pages for the private code and data can appear anywhere in the logical address space.

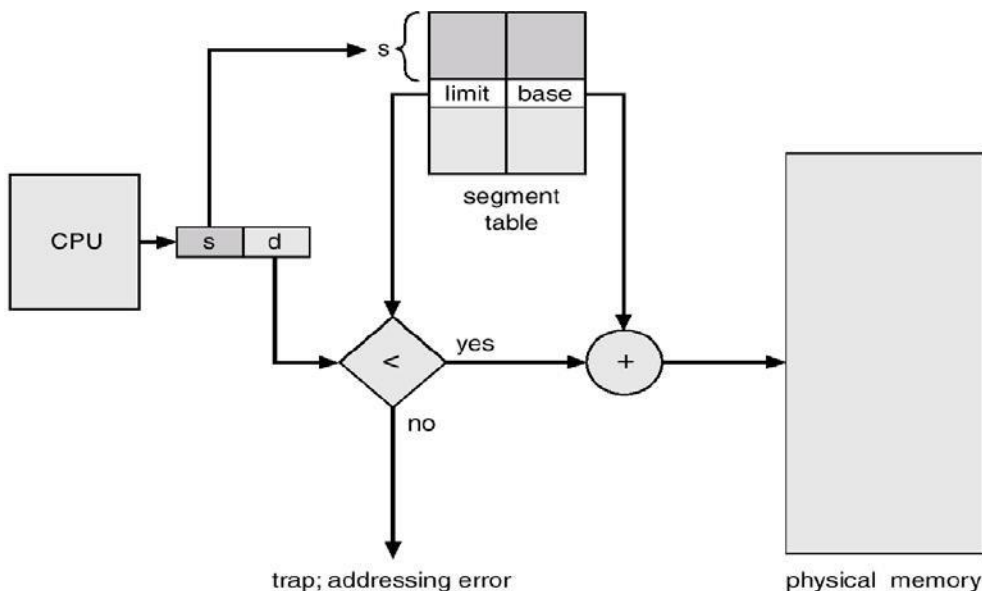
SEGMENTATION

Segmentation is a memory-management scheme that supports user view of memory. A program is a collection of segments. A segment is a logical unit such as: main program, procedure, function, method, object, local variables, global variables, common block, stack, symbol table, arrays etc.

A logical-address space is a collection of segments. Each segment has a name and a length. The user specifies each address by two quantities: a segment name/number and an offset.

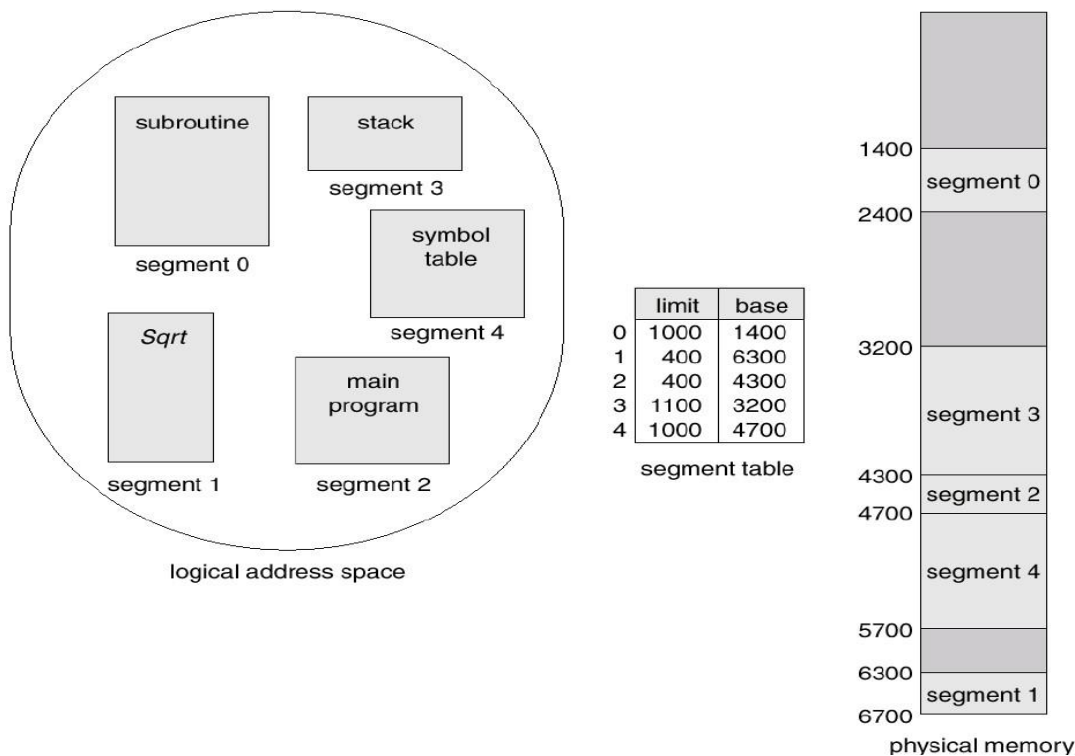
Hence, Logical address consists of a two tuple: $\langle \text{segment-number}, \text{offset} \rangle$ Segment table maps two-dimensional physical addresses and each entry in table has: **base** – contains the starting physical address where the segments reside in memory. **limit** – specifies the length of the segment.

Segment-table base register (STBR) points to the segment table's location in memory. **Segment-table length register (STLR)** indicates number of segments used by a program.



(Diagram of Segmentation Hardware)

The segment number is used as an index into the segment table. The offset d of the logical address must be between 0 and the segment limit. If it is not, we trap to the operating system that logical addressing attempt beyond end of segment. If this offset is legal, it is added to the segment base to produce the address in physical memory of the desired byte. Consider we have five segments numbered from 0 through 4. The segments are stored in physical memory as shown in figure. The segment table has a separate entry for each segment, giving start address in physical memory (or base) and the length of that segment (or limit). For example, segment 2 is 400 bytes long and begins at location 4300. Thus, a reference to byte 53 of segment 2 is mapped onto location $4300 + 53 = 4353$.

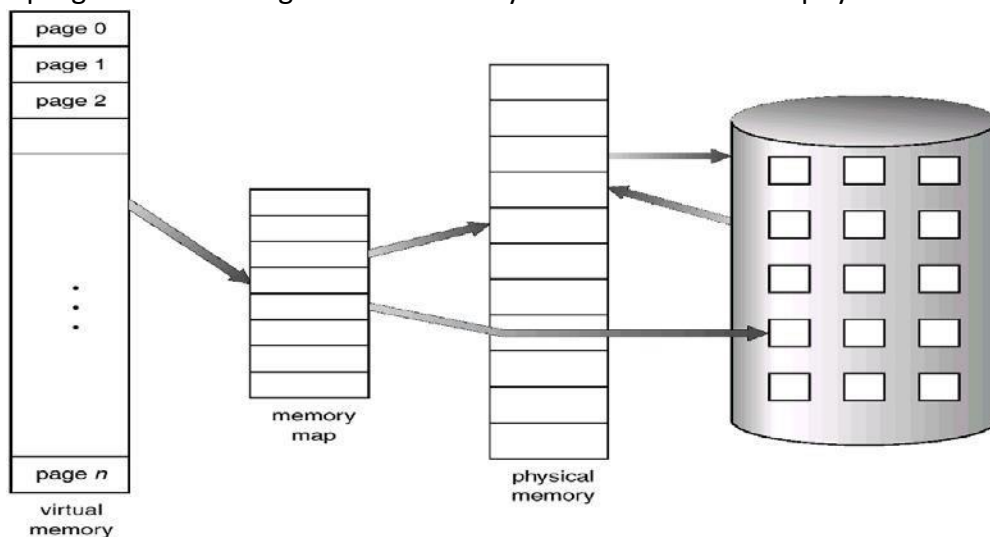


(Example of segmentation)

VIRTUAL MEMORY

Virtual memory is a technique that allows the execution of processes that may not be completely in memory. Only part of the program needs to be in memory for execution. It means that Logical address space can be much larger than physical address space. Virtual memory allows processes to easily share files and address spaces, and it provides an efficient mechanism for process creation.

Virtual memory is the separation of user logical memory from physical memory. This separation allows an extremely large virtual memory to be provided for programmers when only a smaller physical memory is available. Virtual memory makes the task of programming much easier, because the programmer no longer needs to worry about the amount of physical memory available.



(Diagram showing virtual memory that is larger than physical memory)

Virtual memory can be implemented via:

❑ Demand paging ❑

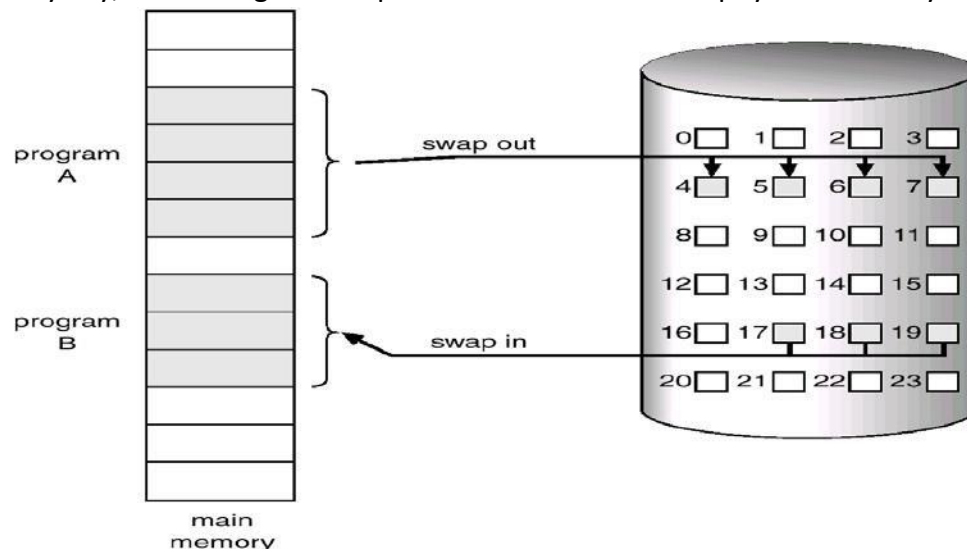
Demand segmentation

DEMAND PAGING

A demand-paging system is similar to a paging system with swapping. Generally, Processes reside on secondary memory (which is usually a disk). When we want to execute a process, we swap it into memory. Rather than swapping the entire process into memory, it swaps the required page. This can be done by a **lazy swapper**.

A lazy swapper never swaps a page into memory unless that page will be needed. A swapper manipulates entire processes, whereas a **pager** is concerned with the individual pages of a process.

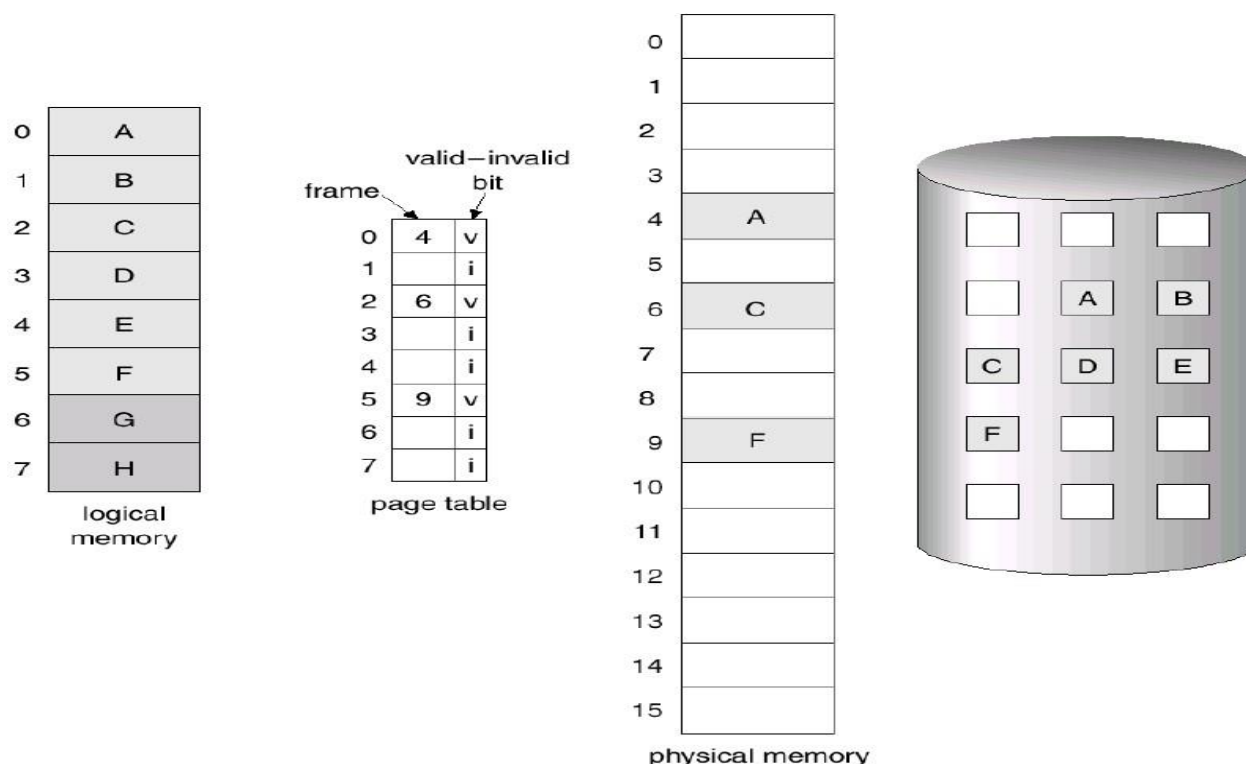
Page transfer Method: When a process is to be swapped in, the pager guesses which pages will be used before the process is swapped out again. Instead of swapping in a whole process, the pager brings only those necessary pages into memory. Thus, it avoids reading into memory pages that will not be used anyway, decreasing the swap time and the amount of physical memory needed.



(Transfer of a paged memory to contiguous disk space) **Page**

Table:

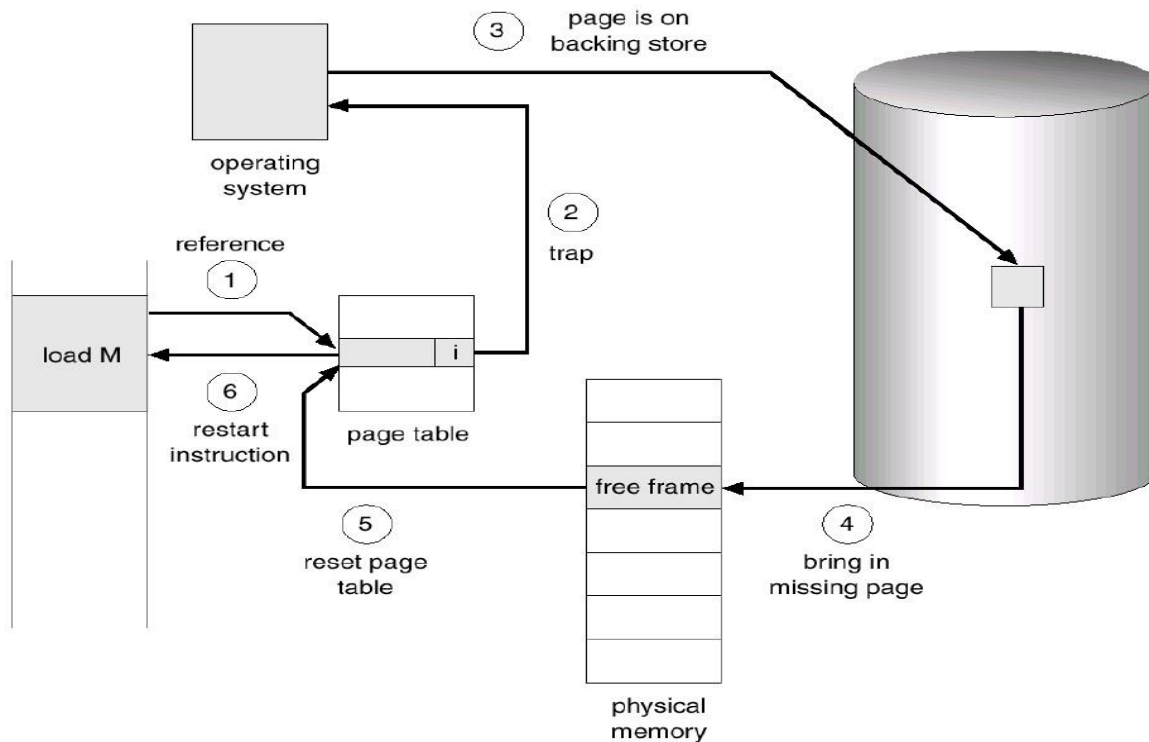
- The valid-invalid bit scheme of Page table can be used for indicating which pages are currently in memory.
- When this bit is set to "valid", this value indicates that the associated page is both legal and in memory. If the bit is set to "invalid", this value indicates that the page either is not valid or is valid but is currently on the disk.
- The page-table entry for a page that is brought into memory is set as usual, but the pagetable entry for a page that is not currently in memory is simply marked invalid, or contains the address of the page on disk.



(Page table when some pages are not in main memory)

When a page references an invalid page, then it is called **Page Fault**. It means that page is not in main memory. The procedure for handling page fault is as follows:

1. We check an internal table for this process, to determine whether the reference was a valid or invalid memory access.
2. If the reference was invalid, we terminate the process. If it was valid, but we have not yet brought in that page in to memory.
3. We find a free frame (by taking one from the free-frame list).
4. We schedule a disk operation to read the desired page into the newly allocated frame.
5. When the disk read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.
6. We restart the instruction that was interrupted by the illegal address trap. The process can now access the page as though it had always been in memory.



(Diagram of Steps in handling a page fault)

Note: The pages are copied into memory, only when they are required. This mechanism is called **Pure Demand Paging**.

Performance of Demand Paging

Let p be the probability of a page fault ($0 \leq p \leq 1$). Then the **effective access time** is

Effective access time = $(1 - p) \times \text{memory access time} + p \times \text{page fault time}$

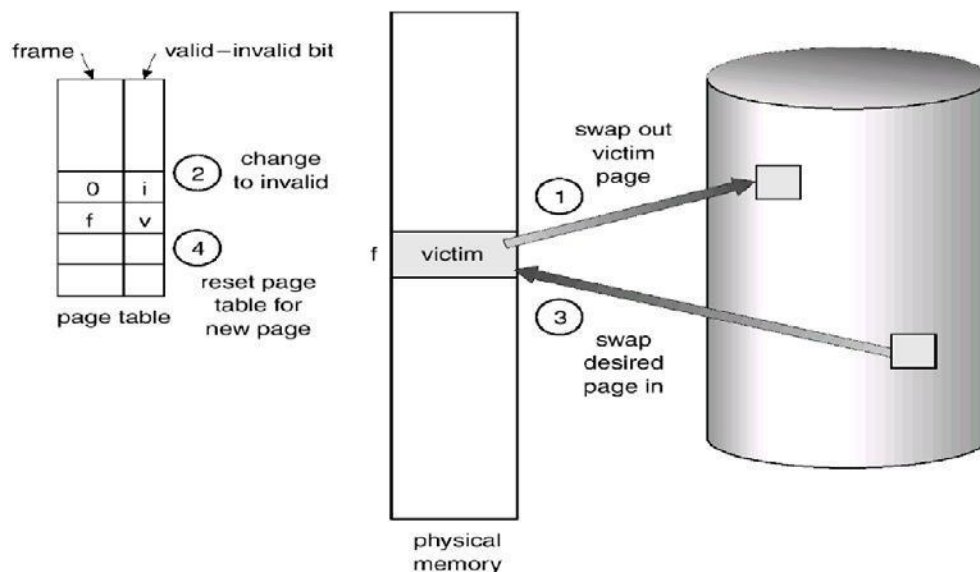
In any case, we are faced with three major components of the page-fault service time:

1. Service the page-fault interrupt.
2. Read in the page.
3. Restart the process.

PAGE REPLACEMENT

The page replacement is a mechanism that loads a page from disk to memory when a page of memory needs to be allocated. Page replacement can be described as follows:

1. Find the location of the desired page on the disk.
2. Find a free frame:
 - a. If there is a free frame, use it.
 - b. If there is no free frame, use a page-replacement algorithm to select a victim frame.
 - c. Write the victim page to the disk; change the page and frame tables accordingly.
3. Read the desired page into the (newly) free frame; change the page and frame tables.
4. Restart the user process.



(Diagram of Page replacement)

Page Replacement Algorithms: The page replacement algorithms decide which memory pages to page out (swap out, write to disk) when a page of memory needs to be allocated. We evaluate an algorithm by running it on a particular string of memory references and computing the number of page faults. The string of memory references is called a **reference string**. The different page replacement algorithms are described as follows:

1. First-In-First-Out (FIFO) Algorithm:

A FIFO replacement algorithm associates with each page the time when that page was brought into memory. When a page must be replaced, the oldest page is chosen to swap out. We can create a FIFO queue to hold all pages in memory. We replace the page at the head of the queue. When a page is brought into memory, we insert it at the tail of the queue. Example:

reference string

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	2		2	2	4	4	4	0			0	0			7	7	7
	0	0	0		3	3	3	2	2	2			1	1			1	0	0
		1	1		1	0	0	0	3	3			3	2			2	2	1

page frames

(FIFO page-replacement algorithm)

Note: For some page-replacement algorithms, the page fault rate may *increase* as the number of allocated frames increases. This most unexpected result is known as **Belady's anomaly**.

2. Optimal Page Replacement algorithm:

One result of the discovery of Belady's anomaly was the search for an **optimal page replacement algorithm**. An optimal page-replacement algorithm has the lowest page-fault rate of all algorithms, and will never suffer from Belady's anomaly. Such an algorithm does exist, and has been called OPT or MIN.

It is simply "Replace the page that will not be used for the longest period of time". Use of this page-replacement algorithm guarantees the lowest possible pagefault rate for a fixed number of frames. Example:

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2		2		2						7		
	0	0	0		0		4		0		0						0		
		1	1		3		3		3		1						1		

page frames

(Optimal page-replacement algorithm)

3. LRU Page Replacement algorithm

If we use the recent past as an approximation of the near future, then we will replace the page that has not been used for the longest period of time. This approach is the **leastrecently-used (LRU)** algorithm.

LRU replacement associates with each page the time of that page's last use. When a page must be replaced, LRU chooses that page that has not been used for the longest period of time.

Example:

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0		1		1		1			
	0	0	0		0		0	0	3	3		3		0		0			
		1	1		3		3	2	2	2		2		2		7			

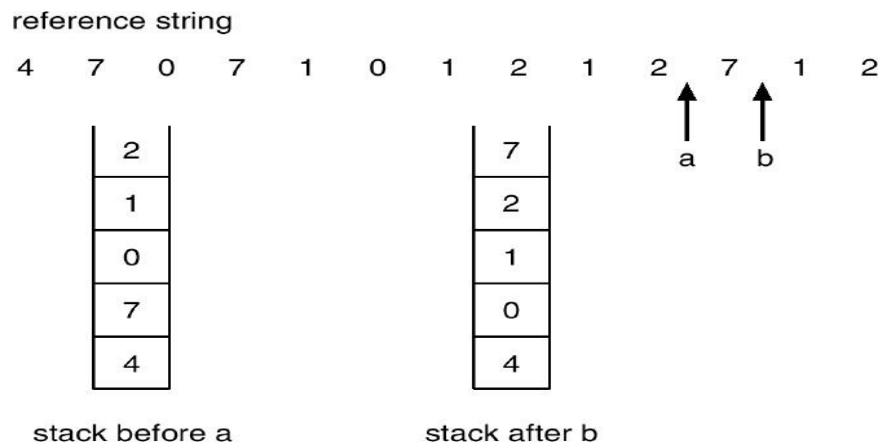
page frames

(LRU page-replacement algorithm)

The major problem is **how** to implement LRU replacement. An LRU page-replacement algorithm may require substantial hardware assistance. The problem is to determine an order for the frames defined by the time of last use. Two implementations are feasible:

Counters: We associate with each page-table entry a time-of-use field, and add to the CPU a logical clock or counter. The clock is incremented for every memory reference. Whenever a reference to a page is made, the contents of the clock register are copied to the time-of-use field in the page-table entry for that page. We replace the page with the smallest time value. This scheme requires a search of the page table to find the LRU page, and a write to memory (to the time-of-use field in the page table) for each memory access. The times must also be maintained when page tables are changed (due to CPU scheduling).

Stack: Another approach to implementing LRU replacement is to keep a stack of page numbers. Whenever a page is referenced, it is removed from the stack and put on the top. In this way, the top of the stack is always the most recently used page and the bottom is the LRU page. Because entries must be removed from the middle of the stack, it is best implemented by a doubly linked list, with a head and tail pointer. Each update is a little more expensive, but there is no search for a replacement; the tail pointer points to the bottom of the stack, which is the LRU page. This approach is particularly appropriate for software or microcode implementations of LRU replacement. Example:



(Use of a stack to record the most recent page references)

4. LRU Approximation Page Replacement algorithm

In this algorithm, Reference bits are associated with each entry in the page table. Initially, all bits are cleared (to 0) by the operating system. As a user process executes, the bit associated with each page referenced is set (to 1) by the hardware. After some time, we can determine which pages have been used and which have not been used by examining the reference bits. This algorithm can be classified into different categories as follows:

i. Additional-Reference-Bits Algorithm: It can keep an 8-bit(1 byte) for each page in a page table in memory. At regular intervals, a timer interrupt transfers control to the operating system. The operating system shifts the reference bit for each page into the highorder bit of its 8-bit, shifting the other bits right over 1 bit position, discarding the low-order bit. These 8 bits shift registers contain the history of page use for the last eight time periods.

If we interpret these 8-bits as unsigned integers, the page with the lowest number is the LRU page, and it can be replaced.

ii. Second-Chance Algorithm: The basic algorithm of second-chance replacement is a FIFO replacement algorithm. When a page has been selected, we inspect its reference bit. If the value is 0, we proceed to replace this page. If the reference bit is set to 1, we give that page a second chance and move on to select the next FIFO page. When a page gets a second chance, its reference bit is cleared and its arrival time is reset to the current time. Thus, a page that is given a second chance will not be replaced until all other pages are replaced.

5. Counting-Based Page Replacement

We could keep a counter of the number of references that have been made to each page, and develop the following two schemes.

i. LFU page replacement algorithm: The **least frequently used (LFU) page-replacement algorithm** requires that the page with the smallest count be replaced. The reason for this selection is that an actively used page should have a large reference count. **ii. MFU page-replacement algorithm:** The **most frequently used (MFU) page replacement algorithm** is based on the argument that the page with the largest count be replaced.

ALLOCATION OF FRAMES

When a page fault occurs, there is a free frame available to store new page into a frame. While the page swap is taking place, a replacement can be selected, which is written to the disk as the user process continues to execute. The operating system allocate all its buffer and table space from the free-frame list for new page.

Two major allocation Algorithm/schemes.

1. equal allocation

2. proportional allocation

1. Equal allocation: The easiest way to split m frames among n processes is to give everyone an equal share, m/n frames. This scheme is called **equal allocation**.

2. proportional allocation: Here, it allocates available memory to each process according to its size. Let the size of the virtual memory for process p_i be s_i , and define $S = \sum s_i$. Then, if the total number of available frames is m , we allocate a_i frames to process p_i , where a_i is approximately $a_i = s_i / S \times m$.

Global Versus Local Allocation

We can classify page-replacement algorithms into two broad categories: **global replacement** and **local replacement**.

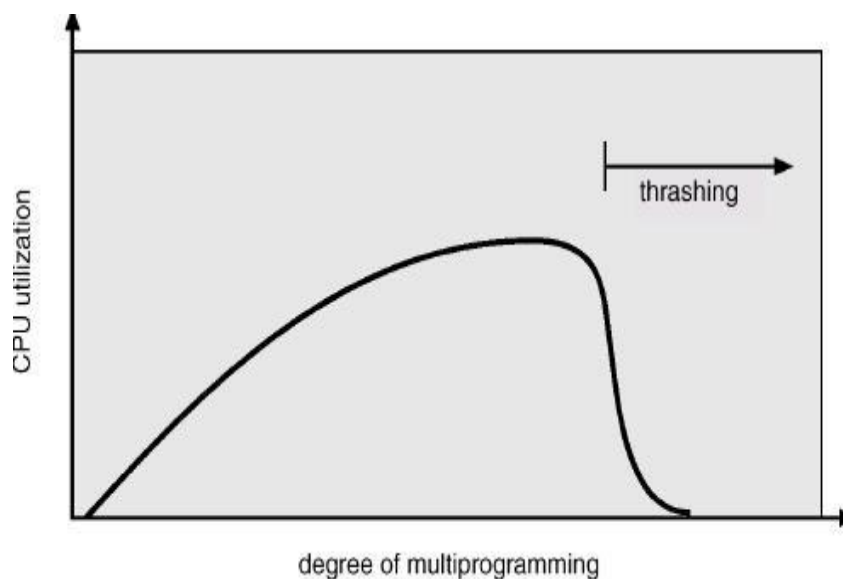
Global replacement allows a process to select a replacement frame from the set of all frames, even if that frame is currently allocated to some other process; one process can take a frame from another.

Local replacement requires that each process select from only its own set of allocated frames.

THRASHING

The system spends most of its time shuttling pages between main memory and secondary memory due to frequent page faults. This behavior is known as thrashing.

A process is thrashing if it is spending more time paging than executing. This leads to: low CPU utilization and the operating system thinks that it needs to increase the degree of multiprogramming.



(Thrashing)